Шинэ Монгол Технологийн Коллеж
Компьютерийн ухааны тэнхим

Оюутны код: s21c011b
Оюутны овог нэр: Үдэлгомбо Тамир

# Байгууллагын бүтээгдэхүүн түгээлтийн нэгдсэн систем
## /ТӨГСӨЛТИЙН СУДАЛГААНЫ АЖИЛ/

Удирдагч багш
Гүйцэтгэсэн оюутан

Н.Соронзонболд
Ү. Тамир

Улаанбаатар хот
2025 он

**Шинэ Монгол Технологийн Коллеж**
**Компьютерийн ухааны тэнхим**

Төгсөлтийн судалгааны ажил

# Байгууллагын бүтээгдэхүүн түгээлтийн нэгдсэн систем

**Гүйцэтгэгч:** Y. Тамир
**Удирдагч:** Н.Соронзонболд

Улаанбаатар хот
2025 он

# Байгууллагын бүтээгдэхүүн түгээлтийн нэгдсэн систем

## Компьютерийн ухааны тэнхим

Ү. Тамир

## Хураангуй

Орчин үед жижиг, дунд бизнесийн байгууллагуудын хооронд бараа бүтээгдэхүүний захиалга, нийлүүлэлтийг хийх үйл явц нь ихэвчлэн захиалагч өөрийн биеэр очиж бараагаа авах хэлбэрээр явагддаг. Энэ нь бизнес эрхлэгчдийн цаг хугацааг их хэмжээгээр алдагдуулж, өдөр тутмын үйл ажиллагаанд хүндрэл учруулдаг.

Иймээс эдгээр асуудлыг шийдвэрлэх, бизнесүүдийн хоорондын бараа нийлүүлэлт болон хүргэлтийн процессыг автоматжуулж, цаг хугацаа болон зардлыг хэмнэх зорилгоор B2B бараа хүргэлтийн интеграцчилсан системийг хөгжүүлэх шаардлага үүссэн.

Энэхүү төслийн гол зорилго нь бизнесүүд хооронд бараа бүтээгдэхүүний захиалга өгөх, баталгаажуулах, хүргүүлэх үйл явцыг нэг цогц платформд нэгтгэсэн, үр ашигтай, хурдан, найдвартай системийг боловсруулахад оршино.

| | |
|---|---|
| **EMS** | **E**mergency **M**edical **S**ervice |
| **EMSS** | **E**mergency **M**edical **S**ervices **S**ystem |
| **VoIP** | **V**oice **O**ver **I**nternet **P**rotocol |
| **PSTN** | **P**ublic **S**witched **T**elephone **N**etwork |
| **IVR** | **I**nteractive **V**oice **R**esponse |
| **CTI** | **C**omputer **T**elephony **I**ntegration |
| **API** | **A**pplication **P**rogramming **I**nterface |
| **STT** | **S**peech **T**o **T**ext |
| **TTS** | **T**ext **T**o **S**peech |
| **ASR** | **A**utomatic **S**peech **R**ecognition |
| **LLM** | **L**arge **L**anguage **M**odel |

# Contents

# 1 Удиртгал

## 1.1 Судалгааны үндэслэл

Мэдээллийн технологийн хурдацтай хөгжил нь бизнесийн бүхий л салбарт, тэр дундаа нийлүүлэлтийн сүлжээний менежментэд (Supply Chain Management — SCM) томоохон өөрчлөлтүүдийг авчирч байна [**chopra2019supply**]. Дэлхий дахинд B2B (Business-to-Business) цахим худалдааны зах зээл жилээс жилд өргөжин тэлж, уламжлалт худалдааны аргуудыг орлож байна. *Grand View Research*-ийн судалгаагаар дэлхийн B2B цахим худалдааны зах зээлийн хэмжээ 2028 он гэхэд 25.6 их наяд ам.долларт хүрэх төлөвтэй байна [**grandview2023**]. Энэхүү өсөлт нь бизнесүүдийг үйл ажиллагаагаа цахимжуулах, зардлаа бууруулах, үр ашгаа нэмэгдүүлэх шаардлагатай нүүр тулгаж байна.

**Монгол улсын нөхцөл байдал** Монгол улсын хувьд жижиг, дунд бизнес (ЖДБ) нь эдийн засгийн томоохон хэсгийг бүрдүүлдэг хэдий ч тэдгээрийн бараа татан авалт, түгээлтийн үйл явц нь ихэвчлэн механик, гар ажиллагаатай хэвээр байна. Үндэсний Статистикийн Хорооны мэдээгээр ЖДБ-үүд нийт аж ахуйн нэгжийн 98%-ийг бүрдүүлдэг [**nso2023** Улаанбаатар хотын замын түгжрэл, хаягжилтын тодорхойгүй байдал, цаг агаарын эрс тэс уур амьсгал зэрэг нь логистикийн үйл ажиллагаанд нэмэлт хүндрэлүүдийг учруулдаг. Одоогийн байдлаар нийлүүлэгч болон захиалагч (дэлгүүр, ресторан, ТҮЦ гэх мэт) хоорондын харилцаа дараах үндсэн хүндрэлүүдтэй тулгарч байна:

**Үндсэн асуудлууд**

1. **Мэдээллийн урсгалын удаашрал ба алдаа:** Захиалгыг утсаар ярих, мессеж бичих, эсвэл худалдааны төлөөлөгч биечлэн очиж авах зэрэг уламжлалт аргуудаар гүйцэтгэж байгаа нь цаг хугацаа их шаарддаг. “Захиалга мартсан”, “Буруу бараа ачсан”, “Тоо хэмжээ зөрсөн” зэрэг алдаанууд нь хүний хүчин зүйлээс шууд хамааралтай байдаг.

2. **Бодит цагийн хяналтгүй байдал** *(Lack of Real-time Visibility):* Захиалагч тал бараагаа хэзээ ирэхийг мэддэггүй, нийлүүлэгч тал хүргэгчээ хаана явж байгааг хянаж чаддаггүй. Энэ нь үл ойлголцол үүсгэж, хэрэглэгчийн сэтгэл ханамжийг бууруулдаг.

3. **Баримт бичгийн эмх замбараагүй байдал:** Цаасан падаан, нэхэмжлэх ашиглах нь баримт бичиг гээгдэх, урагдах, тооцоо нийлэхэд хүндрэл учруулах эрсдэлтэй. Санхүүгийн тооцоог гараар шивэх нь давхар ажил үүсгэдэг.

4. **Маршрутын оновчлолгүй байдал:** Хүргэгчид өөрсдийн туршлагаар замаа сонгодог нь шатахууны зарцуулалтыг нэмэгдүүлж, хүргэлтийн хугацааг уртасгадаг [**toth201**

Эдгээр асуудлууд нь бизнесийн үйл ажиллагааны зардлыг нэмэгдүүлж, бүтээмжийг бууруулж, эцсийн хэрэглэгчид хүрэх үйлчилгээний чанарт сөргөөр нөлөөлж байна. Иймд орчин үеийн гар утасны технологид суурилсан, автоматжуулсан нэгдсэн системийг хөгжүүлж нэвтрүүлэх нь цаг үеийн зайлшгүй шаардлага болж байна [**wang2016bigdata**]. Зураг **??**-д одоогийн системийн гол асуудлуудыг схемчлэн харуулав.

## 1.2 Судалгааны зорилго

Энэхүү дипломын ажлын үндсэн зорилго нь Монгол улсын жижиг, дунд бизнесүүдийн хооронд (B2B) бүтээгдэхүүн захиалах, хүлээн авах, хүргэх үйл явцыг нэгтгэсэн, мэдээллийн технологийн дэвшилтэт шийдэл бүхий **"Tdelivery"** гар утасны аппликейшн системийг зохион бүтээж, хөгжүүлэхэд оршино.
Энэхүү систем нь дараах үндсэн функцуудыг хэрэгжүүлнэ:

- Захиалгын удирдлагыг автоматжуулах
- Хүргэлтийн явцыг бодит цагт хянах
- Цаасан баримтыг цахимжуулах
- Гурван талт (нийлүүлэгч, захиалагч, хүргэгч) нэгдсэн платформ бий болгох

## 1.3 Судалгааны зорилтууд

Дээрх зорилгод хүрэхийн тулд дараах нарийвчилсан зорилтуудыг дэвшүүлж, шийдвэрлэнэ (Хүснэгт **??**-д товчилсон байдлаар харуулав):

1. **Онолын болон практик судалгаа хийх:**
   - Нийлүүлэлтийн сүлжээний удирдлага, "Last-mile delivery" ойлголтуудыг судлах.
   - Одоо ашиглагдаж буй дотоодын болон гадаадын ижил төстэй системүүдэд харьцуулсан шинжилгээ хийж, давуу болон сул талуудыг тодорхойлох.
   - NoSQL өгөгдлийн сангийн бүтэц, React Native фреймворкийн онцлогийг судлах.
2. **Системийн шинжилгээ ба зохиомж:**
   - Хэрэглэгчийн шаардлагыг тодорхойлж, Use Case, Activity, Sequence диаграммуудыг боловсруулах.
   - Системийн архитектур болон өгөгдлийн сангийн (ERD) зохиомжийг гаргах.
   - Хэрэглэгчийн интерфэйс (UI/UX) загварыг гаргах.
3. **Программын хөгжүүлэлт:**
   - React Native ашиглан Захиалагч, Нийлүүлэгч, Хүргэгчдэд зориулсан гар утасны аппликейшнийг хөгжүүлэх.
   - Google Firebase (Authentication, Firestore, Cloud Functions) ашиглан Back-end системийг бүрдүүлэх.
   - Google Maps API ашиглан байршил тогтоох, маршрут харуулах функцуудыг хөгжүүлэх.
4. **Туршилт ба үнэлгээ:**
   - Системийн үйл ажиллагааг тестлэх (Unit test, Integration test).
   - Бодит өгөгдөл дээр туршилт хийж, үр дүнг үнэлэх.

## 1.4 Судалгааны ач холбогдол

Энэхүү судалгааны ажлын хүрээнд хөгжүүлж буй “Tdelivery” систем нь дараах практик болон онолын ач холбогдолтой:

### 1.4.1 Практик ач холбогдол

- **Үйл ажиллагааны үр ашиг:** Захиалга, хүргэлтийн процессыг автоматжуулснаар цаг хугацааг 50–70% хэмнэх, хүний нөөцийн зардлыг бууруулах боломжийг олгоно.
- **Мэдээллийн ил тод байдал:** Захиалагч, нийлүүлэгч, хүргэгч гэсэн гурван тал нэгдсэн платформ дээр бодит цагийн *(Real-time)* мэдээлэл солилцох боломжтой болсноор үл ойлголцол арилж, итгэлцэл нэмэгдэнэ.
- **Байгаль орчинд ээлтэй:** Цаасан баримт бичгийн хэрэглээг халж, цахим баримт *(e-POD — electronic Proof of Delivery)* нэвтрүүлснээр байгаль орчинд ээлтэй “Ногоон логистик”-ийн чиг хандлагад нийцнэ [**greenlogistics2020**].
- **Өгөгдөлд суурилсан шийдвэр гаргалт:** Системийн цуглуулсан өгөгдөлд дүн шинжилгээ хийснээр бизнесийн оновчтой шийдвэр гаргах боломжтой болно.

### 1.4.2 Нийгэм, эдийн засгийн ач холбогдол

- Жижиг дунд бизнес эрхлэгчдийн өрсөлдөх чадварыг нэмэгдүүлнэ.
- Хүргэлтийн үйлчилгээний чанарыг сайжруулснаар эцсийн хэрэглэгчийн сэтгэл ханамж өснө.
- Мэдээллийн технологийн дэвшлийг уламжлалт салбарт нэвтрүүлж, дижитал шилжилтийг хурдасгана.

## 1.5 Судалгааны хамрах хүрээ ба хязгаарлалт

### 1.5.1 Хамрах хүрээ

- **Газарзүйн хүрээ:** Улаанбаатар хотын хэмжээнд үйл ажиллагаа явуулж буй хүнсний болон өргөн хэрэглээний барааны нийлүүлэлтийн сүлжээ.
- **Техникийн хүрээ:** iOS болон Android үйлдлийн системтэй ухаалаг гар утас хэрэглэгчид.
- **Тээврийн хүрээ:** Жижиг оврын ачаа тээвэр (мотоцикл, суудлын тэрэг, портер).
- **Хэрэглэгчийн хүрээ:** Нийлүүлэгч, захиалагч (жижиглэн худалдааны цэгүүд), хүргэгч нар.

### 1.5.2 Хязгаарлалт

- **Сүлжээний хамаарал:** Систем нь зөвхөн интернэт сүлжээтэй орчинд бүрэн ажиллах боломжтой (хэдийгээр оффлайн горим дэмждэг ч өгөгдөл синхрончлоход сүлжээ шаардлагатай).

- **Төлбөрийн систем:** Төлбөр тооцооны систем нь одоогоор зөвхөн бэлэн мөнгө болон дансны шилжүүлгийг бүртгэх хэлбэрээр шийдэгдсэн (Банкны API шууд холболт хийгдээгүй).
- **Ачааны төрөл:** Хүнд даацын болон тусгай зөвшөөрөл шаардлагатай ачаа тээвэрлэлтийг хамраагүй.
- **Хугацааны хязгаар:** Туршилтын үе шат зөвхөн 2–3 сарын хугацаанд хийгдсэн.

## 1.6 Дипломын ажлын бүтэц

Энэхүү дипломын ажил нь удиртгал, 4 үндсэн бүлэг, дүгнэлт, ном зүй, хавсралт гэсэн хэсгүүдээс бүрдэнэ (Зураг **??**).

**Нэгдүгээр бүлэг:** Судалгааны ажлын үндэслэл, зорилго, зорилт, ач холбогдлыг тодорхойлно.

**Хоёрдугаар бүлэг:** Сэдвийн хүрээн дэх онолын судалгаа болон ижил төстэй системүүдийн харьцуулсан шинжилгээг хийнэ.

**Гуравдугаар бүлэг:** Системийн шинжилгээ, зохиомж, архитектурын шийдлийг дэлгэрэнгүй танилцуулна.

**Дөрөвдүгээр бүлэг:** Системийн хэрэгжүүлэлт, ашигласан технологиуд, кодын тайлбар, туршилтын үр дүнг багтаана.

**Дүгнэлт:** Ажлын үр дүнг нэгтгэн дүгнэж, цаашдын хөгжүүлэлтийн саналыг дэвшүүлнэ.

Table 1: Судалгааны зорилтуудын товч

| № | Зорилт | Үр дүн |
|---|---|---|
| 1 | Онолын судалгаа | Харьцуулсан шинжилгээ |
| 2 | Системийн шинжилгээ | UML диаграммууд |
| 3 | Программ хөгжүүлэлт | React Native аппликейшн |
| 4 | Туршилт ба үнэлгээ | Тестийн үр дүн |

# 2 Онолын болон харьцуулсан судалгаа

## 2.1 Онолын судалгаа

Энэ хэсэгт B2B түгээлтийн бизнесийн процесс, логистик систем, маршрутын оновчлол, болон гар утасны хөгжүүлэлтийн технологиудын талаарх онолын ойлголтуудыг дэлгэрэнгүй авч үзнэ.

### 2.1.1 Нийлүүлэлтийн сүлжээний удирдлага (SCM)

Нийлүүлэлтийн сүлжээний удирдлага (Supply Chain Management - SCM) нь түүхий эд бэлтгэн нийлүүлэгчээс эхлээд эцсийн хэрэглэгч хүртэлх бараа, үйлчилгээ, мэдээлэл, санхүүгийн урсгалыг төлөвлөх, хэрэгжүүлэх, хянах цогц үйл ажиллагаа юм. Орчин үед SCM нь зөвхөн ложистик бус, мэдээллийн технологитой нягт уялдаж, "Ухаалаг нийлүүлэлтийн сүлжээ" (Smart Supply Chain) болон хөгжиж байна. SCM-ийн үндсэн зорилго нь нийлүүлэлтийн сүлжээний нийт зардлыг бууруулахын зэрэгцээ хэрэглэгчийн сэтгэл ханамжийг дээд зэргээр хангахад оршино.

**SCM-ийн үндсэн бүрэлдэхүүн хэсгүүд:**

- **Төлөвлөлт (Planning):** Эрэлтийг таамаглах, нөөцийг төлөвлөх.
- **Худалдан авалт (Sourcing):** Нийлүүлэгчийг сонгох, гэрээ байгуулах.
- **Үйлдвэрлэл (Making):** Түүхий эдийг бэлэн бүтээгдэхүүн болгох.
- **Түгээлт (Delivering):** Логистик, агуулах, тээвэрлэлт.
- **Буцаалт (Returning):** Гэмтэлтэй эсвэл илүүдэл барааг буцаах үйл явц.

### 2.1.2 B2B Түгээлтийн процесс ба онцлог

B2B (Business-to-Business) түгээлт нь үйлдвэрлэгчээс бөөний худалдаачид, эсвэл бөөний худалдаачдаас жижиглэн худалдаачид руу бараа нийлүүлэх үйл явц юм. B2C (Business-to-Consumer) хүргэлтээс ялгаатай нь:

- **Захиалгын хэмжээ:** B2B захиалга нь ихэвчлэн тоо хэмжээний хувьд их, овор хэмжээ томтой байдаг.
- **Давтамж:** Тогтмол хуваарийн дагуу, давтагдах шинж чанартай (жишээ нь: долоо хоног бүрийн Даваа гарагт).
- **Харилцаа:** Урт хугацааны гэрээт харилцаанд суурилдаг бөгөөд итгэлцэл чухал үүрэгтэй.
- **Шийдвэр гаргалт:** Олон шат дамжлагатай, оновчтой байдлыг чухалчилдаг. Худалдан авалтын шийдвэрийг сэтгэл хөдлөлөөр бус, үр ашгийн тооцооллоор гаргадаг.

### 2.1.3 Last-Mile Delivery (Сүүлийн цэгийн хүргэлт)

"Last-Mile Delivery" гэдэг нь барааг түгээлтийн төвөөс эцсийн хэрэглэгч (дэлгүүр эсвэл хувь хүн) хүртэл хүргэх логистикийн сүүлийн үе шатыг хэлнэ. Энэ нь нийлүүлэлтийн сүлжээний хамгийн өндөр зардалтай (нийт зардлын 53% хүртэл), хамгийн их цаг хугацаа шаардсан, хамгийн төвөгтэй хэсэгт тооцогддог. Улаанбаатар хотын нөхцөлд замын түгжрэл, хаягжилтын асуудал нь Last-Mile Delivery-ийн үр ашгийг эрс бууруулдаг.

### 2.1.4 Маршрутын оновчлол (Route Optimization)

Хүргэлтийн зардлыг бууруулах, цаг хугацааг хэмнэхэд маршрутын оновчлол чухал үүрэгтэй. Энэ нь "Traveling Salesman Problem" (TSP) буюу олон цэгт хүргэлт хийх хамгийн дөт, үр ашигтай замыг тооцоолох математик аргачлал дээр суурилдаг. Орчин үед Google Maps API, Mapbox зэрэг технологиудыг ашиглан замын түгжрэл, цаг агаар, тээврийн хэрэгслийн даац зэрэг олон хүчин зүйлийг тооцон маршрутыг динамикаар төлөвлөх боломжтой болсон.

### 2.1.5 Электрон баримт бичиг (e-POD)

Electronic Proof of Delivery (e-POD) буюу Цахим хүргэлтийн баталгаажуулалт нь цаасан баримтыг халж, дижитал хэлбэрээр хүргэлтийг баталгаажуулах стандарт юм. e-POD систем нь дараах мэдээллийг агуулдаг:

- Хүлээн авагчийн цахим гарын үсэг.
- Хүргэгдсэн барааны зураг.
- Хүргэлт хийгдсэн газарзүйн байршил (GPS координат).
- Хүргэлт хийгдсэн цаг хугацаа (Timestamp).

Энэ нь маргаантай асуудлыг шийдвэрлэх, төлбөр тооцоог хурдасгахад чухал ач холбогдолтой.

## 2.2 Технологийн судалгаа

Энэхүү төслийг хэрэгжүүлэхэд ашигласан үндсэн технологиудын онцлог, давуу талуудыг судлав. Tdelivery системийн хөгжүүлэлтэд ашигласан технологиудыг дараах ангиллаар танилцуулав. Орчин үеийн гар утасны аппликейшн хөгжүүлэлтэд олон технологи хамтран ажилладаг бөгөөд тус бүрийн үүрэг, ач холбогдлыг дэлгэрэнгүй авч үзье.

### 2.2.1 Програмчлалын хэл

Системийг хөгжүүлэхэд дараах програмчлалын хэлүүдийг ашигласан. Програмчлалын хэл нь аппликейшний суурь болох бөгөөд зөв сонголт хийх нь төслийн амжилтад шийдвэрлэх нөлөөтэй.

Table 2: Програмчлалын хэлүүд

| Технологи | Хувилбар | Тайлбар |
|---|---|---|
| TypeScript | ~5.9.2 | Үндсэн програмчлалын хэл. JavaScript-ийн супер сет бөгөөд статик төрөл шалгалттай. |
| JavaScript (ES6+) | - | React Native-ийн суурь хэл |
| JSX/TSX | - | UI компонентуудыг тодорхойлох синтакс |
| CSS | - | Стиль (Tailwind CSS-ээр дамжуулан) |

**TypeScript гэж юу вэ?**

TypeScript нь Microsoft компаниас 2012 онд хөгжүүлсэн програмчлалын хэл бөгөөд JavaScript-ийн "супер сет" (superset) юм. Энэ нь JavaScript-ийн бүх боломжийг агуулахын зэрэгцээ статик төрөл шалгалт (static type checking), интерфэйс (interfaces), энумераци (enums), женерик (generics) зэрэг нэмэлт боломжуудыг санал болгодог. TypeScript код нь компайл хийгдэж цэвэр JavaScript болдог тул аль ч JavaScript орчинд ажиллах боломжтой.

**TypeScript-ийн давуу талууд:**

- **Статик төрөл шалгалт (Static Type Checking):** Компайл хийх үед алдааг илрүүлж, runtime алдааг багасгадаг. Жишээ нь: `let count: number = "hello"` гэж бичвэл компайл хийх үед алдаа гарна. Энэ нь runtime үед гарах алдааг урьдчилан сэргийлдэг.

- **Код дахин ашиглалт:** Interface болон Type-ууд нь кодын бүтцийг тодорхой болгож, дахин ашиглах боломжийг нэмэгдүүлдэг. Жишээ нь захиалгын бүтцийг нэг удаа тодорхойлоод бүх газар ашиглах боломжтой.

- **IDE дэмжлэг:** VS Code зэрэг орчин үеийн IDE-үүдэд автомат бөглөлт (IntelliSense), алдаа илрүүлэлт, рефакторинг зэрэг боломжуудыг сайн дэмждэг. Энэ нь хөгжүүлэлтийн бүтээмжийг 20-30% нэмэгдүүлдэг гэж үздэг.

- **Том төслийн удирдлага:** Олон хөгжүүлэгчтэй төсөлд кодын чанарыг хадгалахад тустай. Төрлийн тодорхойлолт нь баримт бичиг шиг үүрэг гүйцэтгэдэг.

- **Алдааг эрт илрүүлэх:** JavaScript-д runtime үед гардаг алдаануудын 15% орчимыг компайл хийх үед илрүүлдэг.

**JavaScript (ES6+) ба JSX/TSX**

JavaScript нь вэб болон гар утасны аппликейшн хөгжүүлэлтийн хамгийн түгээмэл хэл бөгөөд ES6 (ECMAScript 2015) хувилбараас хойш олон чухал шинэчлэлтүүд нэмэгдсэн:

- **Arrow Functions:** Богино синтаксээр функц тодорхойлох (`const add = (a, b) => a + b`)

- **Async/Await:** Асинхрон кодыг синхрон маягаар бичих боломж

- **Destructuring:** Объект болон массивын утгуудыг задлах (`const {name, age} = user`)

- **Spread Operator:** Массив, объектыг тараах (`[...arr1, ...arr2]`)
- **Template Literals:** Тэмдэгт мөрийг динамикаар үүсгэх (`‘Hello ${name}‘`)

JSX (JavaScript XML) нь React-д UI компонентуудыг HTML-тэй төстэй синтаксээр бичих боломжийг олгодог. TSX нь TypeScript дээр JSX ашиглах өргөтгөл юм. Жишээ нь:

```
// TSX
const OrderCard: React.FC<{order: Order}> = ({order}) => (
  <View style={styles.card}>
    <Text>{order.customerName}</Text>
    <Text>{order.totalAmount}</Text>
  </View>
);
```

### 2.2.2 Framework ба Platform

React Native болон Expo платформ нь гар утасны аппликейшн хөгжүүлэлтийн үндэс болсон. Эдгээр технологиуд нь нэг кодын баазаар олон платформд ажиллах боломжийг олгодог.

Table 3: Framework ба Platform

| Технологи | Хувилбар | Тайлбар |
|---|---|---|
| React Native | 0.81.5 | Гар утасны апп хөгжүүлэлтийн фреймворк |
| React | 19.1.0 | UI компонент сан |
| Expo | ~54.0.20 | React Native хөгжүүлэлтийн платформ |
| React Native Web | ~0.21.0 | Веб дэмжлэг |

**React Native гэж юу вэ?**

React Native нь Facebook (Meta) компаниас 2015 онд нээлттэй эх болгон гаргасан гар утасны аппликейшн хөгжүүлэлтийн фреймворк юм. React Native нь JavaScript ашиглан бичигдсэн кодыг native компонентууд руу хөрвүүлдэг тул бусад hybrid шийдлүүдээс (Cordova, Ionic) илүү өндөр гүйцэтгэлтэй.

**React Native-ийн ажиллах зарчим:**

1. JavaScript код нь JavaScript Engine (Hermes эсвэл JSC) дээр ажиллана.
2. Bridge (гүүр) нь JavaScript болон Native код хооронд мэдээлэл дамжуулна.
3. Native модулиуд нь төхөөрөмжийн боломжуудад (камер, GPS, гэх мэт) хандана.
4. UI компонентууд нь жинхэнэ native элементүүд рүү хөрвүүлэгдэнэ.

**React Native-ийн давуу талууд:**

- **Cross-Platform (Олон платформ):** Нэг кодын баазаар iOS болон Android үйлдлийн системд зэрэг хөгжүүлэлт хийх боломжтой. Судалгаагаар кодын 80-90%-ийг дахин ашиглах боломжтой. Энэ нь хөгжүүлэлтийн цаг болон зардлыг 40-50% хэмнэдэг.

- **Performance (Гүйцэтгэл):** Native компонентүүдийг ашигладаг тул Hybrid аппликейшнүүдтэй (WebView суурьтай) харьцуулахад 60 FPS гүйцэтгэлтэй, анимаци гөлгөр ажилладаг.
- **Hot Reloading (Халуун дахин ачаалалт):** Кодод өөрчлөлт оруулахад аппликейшнийг дахин ачаалахгүйгээр үр дүнг шууд харах боломжтой. Энэ нь хөгжүүлэлтийн хурдыг 2-3 дахин нэмэгдүүлдэг.
- **Том коммьюнити:** Дэлхий даяар 2 сая гаруй хөгжүүлэгч ашигладаг тул тусламж, бэлэн сангууд (npm дээр 400,000+ package) ихтэй.
- **Код дахин ашиглалт:** Бизнес логик, утилит функцууд, төлөв удирдлага зэрэг кодыг iOS, Android, Web хооронд бүрэн дахин ашиглах боломжтой.

**React гэж юу вэ?**

React нь хэрэглэгчийн интерфэйс (UI) бүтээхэд зориулагдсан JavaScript сан бөгөөд Facebook-аас 2013 онд гаргасан. React-ийн гол онцлог нь компонент суурьтай архитектур юм. UI-г жижиг, дахин ашиглагдах компонентуудад хуваан зохион байгуулдаг.

**React-ийн гол ойлголтууд:**

- **Virtual DOM:** Бодит DOM-ийг шууд өөрчлөхийн оронд виртуал DOM дээр өөрчлөлт хийгээд зөвхөн өөрчлөгдсөн хэсгийг бодит DOM-д шинэчилдэг. Энэ нь гүйцэтгэлийг эрс нэмэгдүүлдэг.
- **Component Lifecycle:** Компонент үүсэх, шинэчлэгдэх, устах үеийн амьдралын мөчлөгийн удирдлага.
- **Hooks:** `useState`, `useEffect`, `useContext` зэрэг функцууд нь функциональ компонентуудад төлөв болон хажуугийн нөлөө (side effects) удирдах боломжийг олгодог.
- **Props ба State:** Props нь эцэг компонентоос хүүхэд компонент руу дамжуулагдах өгөгдөл, State нь компонентын дотоод төлөв юм.

**Expo гэж юу вэ?**

Expo нь React Native дээр суурилсан платформ бөгөөд гар утасны аппликейшн хөгжүүлэлтийг илүү хялбар болгодог. Expo нь "managed workflow" болон "bare workflow" гэсэн хоёр төрлийн хөгжүүлэлтийн загварыг санал болгодог.

**Expo-ийн давуу талууд:**

- **Хялбар эхлэл:** Xcode (iOS) эсвэл Android Studio (Android) суулгахгүйгээр хөгжүүлэлт эхлүүлэх боломжтой. `npx create-expo-app` командаар шинэ төсөл үүсгэхэд 2 минут хангалттай.
- **Expo Go апп:** Бодит төхөөрөмж дээр шууд туршилт хийх боломжтой. QR код уншуулахад л хангалттай.
- **Бэлэн SDK:** Камер, байршил (GPS), мэдэгдэл (Push Notifications), файл систем, биометрик баталгаажуулалт зэрэг 50 гаруй бэлэн модуль байдаг. Энэ нь native код бичих шаардлагагүй болгодог.
- **OTA Updates (Over-The-Air):** App Store/Google Play-д дахин илгээхгүйгээр JavaScript кодын шинэчлэлтийг хэрэглэгчдэд шууд хүргэх боломжтой. Энэ нь засвар хийх хугацааг хэдэн өдрөөс хэдэн минут болгон багасгадаг.

- **EAS Build:** Үүлэн дээр build хийх боломжтой тул локал машинд Xcode/Android Studio суулгах шаардлагагүй.

- **Expo Router:** Файлын бүтцэд суурилсан навигаци систем нь Next.js-тэй төстэй routing боломжийг санал болгодог.

**React Native Web**

React Native Web нь React Native компонентуудыг вэб хөтөч дээр ажиллуулах боломжийг олгодог. Энэ нь нэг кодоор гар утас (iOS, Android) болон вэб аппликейшн хөгжүүлэх боломжийг бүрдүүлдэг. Twitter, Uber Eats зэрэг томоохон компаниуд энэ аргачлалыг ашигладаг.

### 2.2.3 Database ба Backend (Firebase)

Firebase нь Google-ийн үүлэн тооцооллын платформ бөгөөд гар утас болон вэб аппликейшн хөгжүүлэхэд зориулагдсан цогц шийдэл юм. "Backend-as-a-Service" (BaaS) загвараар ажилладаг бөгөөд энэ нь сервер талын инфраструктурыг бүрэн удирдах шаардлагагүй болгодог.

Table 4: Firebase үйлчилгээнүүд

| **Үйлчилгээ** | **Зориулалт** |
|---|---|
| Firebase (v12.4.0) | Backend-as-a-Service платформ |
| Cloud Firestore | NoSQL өгөгдлийн сан, бодит цагийн синхрончлол |
| Firebase Authentication | Хэрэглэгчийн баталгаажуулалт (Утас, Email) |
| Firebase Storage | Файл хадгалах (зураг гэх мэт) |
| Firebase Realtime Database | Бодит цагийн өгөгдөл |

**Firebase гэж юу вэ?**

Firebase нь анх 2011 онд стартап компани болон байгуулагдсан бөгөөд 2014 онд Google компани худалдан авсан. Өнөөдөр Firebase нь Google Cloud Platform-ийн нэг хэсэг бөгөөд дэлхий даяар 3 сая гаруй апп ашигладаг. Firebase нь "Serverless" архитектурыг дэмждэг тул хөгжүүлэгч сервер удирдах, scale хийх зэрэг асуудлуудаар толгой өвдөхгүй.

**Cloud Firestore - NoSQL Өгөгдлийн сан**

Cloud Firestore нь Firebase-ийн хамгийн чухал үйлчилгээ бөгөөд NoSQL документ-суурьтай (document-oriented) өгөгдлийн сан юм. Firestore нь өгөгдлийг collection (цуглуулга) болон document (баримт) бүтцээр хадгалдаг.

**Firestore-ийн өгөгдлийн бүтэц:**

```
// Firestore
orders (collection)
  |-- orderId1 (document)
  |      |-- customerId: "cust123"
  |      |-- items: [{name: "Product A", qty: 10}]
  |      |-- status: "pending"
  |      |-- createdAt: Timestamp
  |-- orderId2 (document)
         |-- ...
```

**Cloud Firestore-ийн онцлог:**

- **Real-time синхрончлол:** Өгөгдөл өөрчлөгдөхөд бүх холбогдсон клиентүүдэд шууд мэдэгддэг. `onSnapshot()` функцээр бодит цагийн сонсогч (listener) тохируулж болно. Энэ нь захиалгын төлөв өөрчлөгдөхөд хэрэглэгчид шууд мэдэгдэх боломжийг олгодог.

- **Offline Persistence (Оффлайн дэмжлэг):** Интернэт холболтгүй үед ч аппликейшн ажиллах чадвартай. Өгөгдөл локал кэшэд хадгалагдаж, холболт сэргэхэд автоматаар синхрончлогддог. Энэ нь хүргэлтийн жолооч интернэтгүй газарт явах үед маш чухал.

- **Scalability (Өргөтгөх чадвар):** Хэрэглэгчийн тоо нэмэгдэхэд автоматаар scale хийгддэг. Google-ийн дэд бүтцийг ашигладаг тул сая сая хэрэглэгчид үйлчлэх боломжтой.

- **Security Rules (Аюулгүй байдлын дүрмүүд):** Нарийвчилсан хандалтын эрхийн удирдлага. Хэн ямар өгөгдөлд хандаж болохыг тодорхойлох боломжтой. Жишээ нь: хэрэглэгч зөвхөн өөрийн захиалгыг үзэх боломжтой.

- **Query боломжууд:** Compound query (олон нөхцөлтэй хайлт), ordering, pagination зэрэг боломжууд бий. Гэхдээ SQL шиг JOIN үйлдэл хийх боломжгүй.

**Firebase Authentication - Баталгаажуулалт**

Firebase Authentication нь хэрэглэгчийн бүртгэл, нэвтрэлтийг удирдах бэлэн шийдэл юм. Дараах аргуудыг дэмждэг:

- **Имэйл/Нууц үг:** Уламжлалт бүртгэлийн арга

- **Утасны дугаар (SMS):** OTP код илгээж баталгаажуулах. Монголд маш түгээмэл.

- **Social Login:** Google, Facebook, Apple, Twitter зэрэг нийгмийн сүлжээгээр нэвтрэх

- **Anonymous:** Бүртгэлгүйгээр түр зуурын хэрэглэгч үүсгэх

Tdelivery системд утасны дугаараар баталгаажуулалт (Phone Authentication) ашигласан. Энэ нь Монголын хэрэглэгчдэд илүү тохиромжтой - имэйл хаяг байнга ашигладаггүй хүмүүст ч хялбар.

**Firebase Storage - Файл хадгалалт**

Firebase Storage нь зураг, видео, аудио зэрэг файлуудыг үүлэн дээр хадгалах үйлчилгээ юм. Google Cloud Storage дээр суурилсан тул өндөр найдвартай, хурдан байдаг. Tdelivery системд дараах зориулалтаар ашигласан:

- Бүтээгдэхүүний зураг хадгалах

- Хүргэлтийн баталгаажуулалтын зураг (e-POD)

- Хэрэглэгчийн профайл зураг

**Firebase-ийн үнийн бодлого**

Firebase нь "Spark" (үнэгүй) болон "Blaze" (pay-as-you-go) гэсэн хоёр үнийн төлөвлөгөөтэй. Spark төлөвлөгөө нь жижиг төсөл, сурах зориулалтаар хангалттай:

- Firestore: 1GB хадгалалт, 50,000 унших, 20,000 бичих өдөрт

- Authentication: Хязгааргүй хэрэглэгч
- Storage: 5GB хадгалалт

### 2.2.4 NoSQL ба Atomic Denormalization

Уламжлалт RDBMS (Relational Database Management System) -ээс ялгаатай нь NoSQL өгөгдлийн сан нь өгөгдлийг баримт (Document) хэлбэрээр хадгалдаг. Энэ хэсэгт NoSQL-ийн онцлог, давуу тал, сул тал болон Firestore-д ашигладаг denormalization аргачлалыг дэлгэрэнгүй авч үзнэ.

**SQL vs NoSQL харьцуулалт**

Table 5: SQL ба NoSQL харьцуулалт

| **Шалгуур** | **SQL (RDBMS)** | **NoSQL (Document DB)** |
|---|---|---|
| Өгөгдлийн бүтэц | Хүснэгт, мөр, багана (Table, Row, Column) | Collection, Document, Field |
| Схем | Хатуу схемтэй (Rigid Schema) | Уян хатан схем (Flexible Schema) |
| Холбоос | JOIN үйлдлээр холбогдсон хүснэгтүүд | Embedded documents, References |
| Scale | Вертикал (илүү хүчтэй сервер) | Хоризонтал (илүү олон сервер) |
| ACID | Бүрэн дэмждэг | Хязгаарлагдмал (document түвшинд) |
| Хэрэглээ | Банк, санхүү, нарийн холбоостой өгөгдөл | Вэб апп, гар утас, бодит цагийн апп |

**NoSQL-ийн давуу талууд гар утасны аппд:**

- **Уян хатан бүтэц:** Схем өөрчлөхөд migration хийх шаардлагагүй. Шинэ field нэмэхэд хуучин өгөгдөлд нөлөөлөхгүй.
- **Бодит цагийн синхрончлол:** WebSocket-ээр өгөгдөл шууд push хийгддэг.
- **Оффлайн дэмжлэг:** Локал кэш, conflict resolution бэлэн.
- **JSON формат:** JavaScript/TypeScript-тэй шууд нийцтэй.

**Atomic Denormalization гэж юу вэ?**

Firebase Firestore-д өгөгдлийн унших хурдыг нэмэгдүүлэхийн тулд "Denormalization" аргыг ашигладаг. Энэ нь нэг өгөгдлийг олон газар хувилж хадгалах арга бөгөөд JOIN үйлдэл хийх шаардлагагүй болгож, query-ийн хурдыг эрс нэмэгдүүлдэг.

**Denormalization-ийн жишээ:**

SQL-д захиалгын мэдээлэл авахад 3 хүснэгтийг JOIN хийх шаардлагатай:

```sql
-- SQL: 3                    JOIN
SELECT o.*, c.name, c.phone, p.name as product_name
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN products p ON o.product_id = p.id
WHERE o.id = 'order123';
```

NoSQL (Firestore)-д denormalization ашиглан нэг document-д бүх мэдээллийг хадгална:

```
// Firestore: Denormalized document
{
  "orderId": "order123",
  "customer": {
    "id": "cust456",
    "name": "         ",         //
    "phone": "99001122"    //
  },
  "product": {
    "id": "prod789",
    "name": "Coca-Cola 1.5L", //
    "price": 3500
  },
  "quantity": 10,
  "totalAmount": 35000
}
```

**Atomic Update - Өгөгдлийн нэгдмэл байдал хангах**

Denormalization-ийн сул тал нь өгөгдөл олон газар хуулбарлагдсан тул нэг газар өөрчлөхөд бүх газар өөрчлөх шаардлагатай. Үүнийг "Atomic Update" буюу batch/-transaction ашиглан шийддэг:

```
// Batch write -
const batch = writeBatch(db);

//
batch.update(doc(db, 'products', 'prod789'), {
  name: 'Coca-Cola 2L'
});

//

orderIds.forEach(orderId => {
  batch.update(doc(db, 'orders', orderId), {
    'product.name': 'Coca-Cola 2L'
  });
});

await batch.commit(); //
```

**Denormalization-ийн зарчмууд:**

1. **Унших давтамж > Бичих давтамж:** Ихэвчлэн уншигддаг өгөгдлийг denormalize хийх нь илүү үр ашигтай.
2. **Хамтдаа хэрэглэгддэг өгөгдөл:** Нэг дэлгэц дээр харуулах өгөгдлийг нэг document-д хадгалах.
3. **Бага өөрчлөгддөг өгөгдөл:** Хэрэглэгчийн нэр, утас зэрэг бага өөрчлөгддөг өгөгдлийг хуулбарлах нь аюулгүй.

### 2.2.5 Навигаци (Navigation)

Аппликейшний дэлгэцүүдийн хооронд шилжихэд React Navigation санг ашигласан. Навигаци нь гар утасны аппликейшний хэрэглэгчийн туршлагын (UX) чухал хэсэг бөгөөд зөв зохион байгуулалт нь аппын ашиглахад хялбар байдлыг тодорхойлдог.

Table 6: Навигацийн сангууд

| Сан | Хувилбар | Зориулалт |
|---|---|---|
| @react-navigation/native | ^7.1.8 | Үндсэн навигаци |
| @react-navigation/native-stack | ^7.1.9 | Stack навигаци (дэлгэц дээр дэлгэц нэмэх) |
| @react-navigation/bottom-tabs | ^7.4.0 | Доод таб навигаци |

**React Navigation гэж юу вэ?**

React Navigation нь React Native аппликейшнд навигаци хийхэд хамгийн өргөн хэрэглэгддэг сан юм. Энэ нь JavaScript дээр бичигдсэн тул cross-platform бөгөөд iOS, Android хоёуланд нэг кодоор ажилладаг.

**Навигацийн төрлүүд:**

- **Stack Navigator:** Дэлгэцүүдийг "stack" (овоолго) шиг удирддаг. Шинэ дэлгэц нээхэд дээр нь нэмэгдэж, буцах товч дарахад дээрээс хасагдана. Ихэнх апп-ийн үндсэн навигаци.
- **Bottom Tab Navigator:** Дэлгэцийн доод хэсэгт байрлах табууд. Үндсэн функцуудад хурдан хандах боломжтой. Tdelivery системд: Нүүр, Захиалга, Сагс, Профайл гэсэн 4 таб ашигласан.
- **Drawer Navigator:** Хажуугаас гулсаж гарах цэс. Олон үйлдэлтэй аппуудад тохиромжтой.
- **Top Tab Navigator:** Дээд хэсэгт байрлах табууд. Material Design-д түгээмэл.

**Tdelivery системийн навигацийн бүтэц:**

```
//
RootNavigator
  |-- AuthStack (                          )
  |      |-- LoginScreen
  |      |-- RegisterScreen
  |      |-- OTPVerificationScreen
  |
  |-- MainTabs (                        )
        |-- HomeStack
        |      |-- HomeScreen
        |      |-- ProductDetailScreen
        |      |-- CategoryScreen
        |
        |-- OrdersStack
        |      |-- OrderListScreen
        |      |-- OrderDetailScreen
        |
        |-- CartStack
        |      |-- CartScreen
        |      |-- CheckoutScreen
        |
        |-- ProfileStack
              |-- ProfileScreen
              |-- EditProfileScreen
              |-- SettingsScreen
```

**Navigation-ийн чухал ойлголтууд:**

- **Screen:** Бие даасан дэлгэц, компонент
- **Route:** Дэлгэц рүү очих зам, параметрүүд
- **Navigator:** Дэлгэцүүдийг удирдах контейнер
- **Navigation Prop:** Дэлгэц хооронд шилжих функцууд (`navigate`, `goBack`, `push`)
- **Route Prop:** Дэлгэцэнд дамжуулсан параметрүүд (`route.params`)

### 2.2.6 UI/UX Сангууд

Хэрэглэгчийн интерфэйсийг сайжруулахад дараах сангуудыг ашигласан. UI (User Interface) нь хэрэглэгчийн харж буй визуал элементүүд, UX (User Experience) нь хэрэглэгчийн аппыг ашиглах туршлага юм.

Table 7: UI/UX сангууд

| **Сан** | **Хувилбар** | **Зориулалт** |
|---|---|---|
| React Native Paper | ^5.12.3 | Material Design UI компонентууд |
| @expo/vector-icons | ^15.0.3 | Дүрс тэмдэг (Icons) |
| NativeWind | ^4.2.1 | Tailwind CSS React Native-д |
| TailwindCSS | ^3.4.18 | Utility-first CSS фреймворк |
| Expo Image | ~3.0.10 | Зураг харуулах, кэшлэх |

**React Native Paper**

React Native Paper нь Google-ийн Material Design дизайны хэлийг React Native-д хэрэгжүүлсэн UI компонент сан юм. 30 гаруй бэлэн компонент агуулдаг:

- **Button:** Primary, outlined, text, contained төрлүүдтэй товч
- **Card:** Бүтээгдэхүүн, захиалга харуулахад тохиромжтой карт
- **TextInput:** Текст оруулах талбар, validation дэмжлэгтэй
- **Snackbar:** Түр зуурын мэдэгдэл (toast)
- **Dialog:** Модал цонх
- **FAB (Floating Action Button):** Хөвөгч үйлдлийн товч
- **ActivityIndicator:** Ачаалж байгааг харуулах индикатор

**Material Design-ийн давуу талууд:**

- Google-ийн дизайны стандарт тул хэрэглэгчдэд танил
- Accessibility (хүртээмжтэй байдал) бэлэн хийгдсэн
- Theming (өнгө, фонт тохируулах) боломжтой

- iOS болон Android дээр нэгдмэл харагдах

**NativeWind ба TailwindCSS**

TailwindCSS нь "utility-first" CSS фреймворк бөгөөд NativeWind нь үүнийг React Native-д ашиглах боломжийг олгодог. Уламжлалт CSS-ээс ялгаатай нь бэлэн class-уудыг шууд ашиглана.

**Уламжлалт StyleSheet vs NativeWind харьцуулалт:**

```
//                      React Native StyleSheet
const styles = StyleSheet.create({
  container: {
    padding: 16,
    backgroundColor: 'white',
    borderRadius: 8,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
  },
  title: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#1f2937',
  }
});

<View style={styles.container}>
  <Text style={styles.title}>                </Text>
</View>
```

```
// NativeWind (TailwindCSS)
<View className="p-4 bg-white rounded-lg shadow-md">
  <Text className="text-lg font-bold text-gray-800">                </
   Text>
</View>
```

**NativeWind/TailwindCSS-ийн давуу талууд:**

- **Utility-first:** Бэлэн class-уудаар хурдан стиль хийх боломжтой. `p-4` нь padding: 16px, `bg-white` нь цагаан дэвсгэр.
- **Responsive:** Дэлгэцийн хэмжээнд тохируулсан стиль. `md:p-8` нь дунд дэлгэц дээр илүү том padding.
- **Consistency:** Урьдчилан тодорхойлсон утгууд (spacing, colors) нь дизайны нэгдмэл байдлыг хангах.
- **Жижиг bundle:** Зөвхөн ашигласан стилүүд bundle-д орно (PurgeCSS).
- **Dark mode:** `dark:bg-gray-900` гэх мэтээр хар горимыг хялбар тохируулах.

**@expo/vector-icons**

Expo Vector Icons нь 10+ icon set-ийг агуулсан сан юм:

- **MaterialIcons:** Google-ийн Material Design icons
- **FontAwesome:** Хамгийн түгээмэл icon сан
- **Ionicons:** iOS-д зориулсан icons

- **Feather:** Хялбар, цэвэрхэн icons

```
import { MaterialIcons, FontAwesome } from '@expo/vector-icons';

<MaterialIcons name="shopping-cart" size={24} color="blue" />
<FontAwesome name="user" size={20} color="#333" />
```

**Expo Image**
Expo Image нь React Native-ийн үндсэн Image компонентоос илүү хүчирхэг:

- **Automatic caching:** Зураг автоматаар кэшлэгдэнэ
- **Progressive loading:** Бүдэг зургаас тод зураг руу шилжих
- **Placeholder:** Ачаалж байх үеийн түр зураг
- **BlurHash:** Зургийн өнгөний товч дүрслэл
- **WebP support:** Шахагдсан зургийн формат дэмжлэг

### 2.2.7 Expo SDK Сангууд

Expo платформын бэлэн сангуудыг ашиглан төхөөрөмжийн боломжуудад хандсан. Expo SDK нь native код бичихгүйгээр төхөөрөмжийн hardware болон software боломжуудад хандах боломжийг олгодог.

Table 8: Expo SDK сангууд

| Сан | Хувилбар | Зориулалт |
|---|---|---|
| expo-constants | ~18.0.10 | Апп-ийн тогтмол утгууд |
| expo-font | ~14.0.9 | Фонт удирдлага |
| expo-haptics | ~15.0.7 | Чичиргээ (Haptic feedback) |
| expo-image-picker | ^17.0.8 | Зураг сонгох |
| expo-linking | ~8.0.8 | Deep linking |
| expo-splash-screen | ~31.0.10 | Splash дэлгэц |
| expo-status-bar | ~3.0.8 | Статус бар удирдлага |
| expo-web-browser | ^15.0.8 | Веб хөтөч нээх |

**Expo SDK сангуудын дэлгэрэнгүй тайлбар:**
**expo-constants** - Апп болон төхөөрөмжийн мэдээлэлд хандах:

```
import Constants from 'expo-constants';

//        -
console.log(Constants.expoConfig.version); // "1.0.0"
//
console.log(Constants.deviceName); // "iPhone 14"
//
console.log(Constants.platform); // { ios: {...} }
```

**expo-font** - Custom фонт ачаалах:

```
import { useFonts } from 'expo-font';

const [fontsLoaded] = useFonts({
  'Roboto-Bold': require('./assets/fonts/Roboto-Bold.ttf'),
  'Roboto-Regular': require('./assets/fonts/Roboto-Regular.ttf'),
});

//
<Text style={{ fontFamily: 'Roboto-Bold' }}>          </Text>
```

**expo-haptics** - Чичиргээ (Haptic feedback):

Хэрэглэгч товч дарах, swipe хийх зэрэг үйлдэл хийхэд физик мэдрэмж өгөх:

```
import * as Haptics from 'expo-haptics';

//
const handlePress = () => {
  Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
};

//
const handleError = () => {
  Haptics.notificationAsync(Haptics.NotificationFeedbackType.Error);
};

//
const handleSuccess = () => {
  Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
};
```

**expo-image-picker** - Зураг сонгох:

Галерей эсвэл камераас зураг авах. Tdelivery системд бүтээгдэхүүний зураг, хүргэлтийн баталгаажуулалтын зураг авахад ашигласан:

```
import * as ImagePicker from 'expo-image-picker';

const pickImage = async () => {
  //
  const { status } = await ImagePicker.
   requestMediaLibraryPermissionsAsync();

  if (status !== 'granted') {
    alert('
                              ');
    return;
  }

  //
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [4, 3],
    quality: 0.8,
  });

  if (!result.canceled) {
    console.log(result.assets[0].uri);
  }
};
```

**expo-linking** - Deep linking:

Бусад апп эсвэл вэб хуудаснаас аппыг нээх, тодорхой дэлгэц рүү шилжих:

```
import * as Linking from 'expo-linking';

//
Linking.openURL('tel:+97699001122');

// Email
Linking.openURL('mailto:support@tdelivery.mn');

// Google Maps
Linking.openURL('https://maps.google.com/?q=47.9184,106.9177');

// Deep link
const url = Linking.createURL('order/12345');
// tdelivery://order/12345
```

**expo-splash-screen** - Splash дэлгэц:

Апп ачаалж байх үед харуулах эхний дэлгэц:

```
import * as SplashScreen from 'expo-splash-screen';

// Splash
SplashScreen.preventAutoHideAsync();

//
useEffect(() => {
  async function prepare() {
    await loadFonts();
    await fetchInitialData();
    await SplashScreen.hideAsync();
  }
  prepare();
}, []);
```

**expo-status-bar** - Статус бар удирдлага:

Дэлгэцийн дээд хэсэгт байрлах статус бар (цаг, батарей, сүлжээ)-ын харагдах байдлыг удирдах:

```
import { StatusBar } from 'expo-status-bar';

//
<StatusBar style="light" />

//
<StatusBar style="dark" />

//                (                                    )
<StatusBar style="auto" />
```

### 2.2.8 State Management ба Storage

Аппликейшний төлөв удирдлага болон локал хадгалалтад дараах шийдлүүдийг ашигласан. State Management нь аппликейшний өгөгдлийг (төлвийг) компонентууд хооронд хуваалцах, синхрончлох үйл явц юм.

Table 9: State Management ба Storage

| Сан | Хувилбар | Зориулалт |
|---|---|---|
| React Context API | - | Төлөв удирдлага (AuthContext, CartContext) |
| @react-native-async-storage/async-storage | ^2.2.0 | Локал хадгалалт |

**State Management гэж юу вэ?**

Гар утасны аппликейшнд олон төрлийн өгөгдөл (төлөв) байдаг:

- **Local State:** Нэг компонентын дотоод төлөв (жишээ нь: form input-ийн утга)
- **Global State:** Олон компонентод хэрэглэгддэг төлөв (жишээ нь: нэвтэрсэн хэрэглэгчийн мэдээлэл)
- **Server State:** Серверээс ирсэн өгөгдөл (жишээ нь: бүтээгдэхүүний жагсаалт)
- **URL State:** Navigation параметрүүд

**React Context API**

React Context API нь React-ийн суурь боломж бөгөөд global state удирдахад хялбар шийдэл юм. Redux, MobX зэрэг гуравдагч талын сангуудаас хялбар, жижигдүнд хэмжээний аппуудад тохиромжтой.

**Context-ийн ажиллах зарчим:**

1. **Context үүсгэх:** `createContext()` функцээр context үүсгэнэ
2. **Provider:** Context-ийн утгыг хангах компонент
3. **Consumer:** Context-ийн утгыг ашиглах компонент (`useContext` hook)

**Tdelivery системийн Context-үүд:**

```
// AuthContext -
const AuthContext = createContext<AuthContextType | null>(null);

interface AuthContextType {
  user: User | null;
  isLoading: boolean;
  signIn: (phone: string, otp: string) => Promise<void>;
  signOut: () => Promise<void>;
}

export const AuthProvider: React.FC = ({ children }) => {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  // Firebase Auth-
  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (firebaseUser) => {
      setUser(firebaseUser);
      setIsLoading(false);
    });
    return unsubscribe;
  }, []);

```

```
  const signIn = async (phone: string, otp: string) => {
    //
  };

  const signOut = async () => {
    await auth.signOut();
  };

  return (
    <AuthContext.Provider value={{ user, isLoading, signIn, signOut }}>
      {children}
    </AuthContext.Provider>
  );
};

// Hook
export const useAuth = () => useContext(AuthContext);
```

```
// CartContext -
const CartContext = createContext<CartContextType | null>(null);

interface CartItem {
  productId: string;
  name: string;
  price: number;
  quantity: number;
}

interface CartContextType {
  items: CartItem[];
  addItem: (product: Product) => void;
  removeItem: (productId: string) => void;
  updateQuantity: (productId: string, quantity: number) => void;
  clearCart: () => void;
  totalAmount: number;
  itemCount: number;
}
```

**React Context API-ийн давуу талууд:**

- **Суурь сан:** Нэмэлт сан суулгах шаардлагагүй, React-д бэлэн байдаг.
- **Prop drilling шийдэл:** Олон түвшний компонентуудаар дамжуулахгүйгээр өгөгдөл дамжуулах. Жишээ нь: App -> Header -> UserMenu гэсэн бүтцэд Header-ээр user дамжуулах шаардлагагүй.
- **Хялбар:** Redux-ээс хялбар, boilerplate код бага. Redux-д 5-6 файл үүсгэх шаардлагатай бол Context-д 1 файл хангалттай.
- **TypeScript дэмжлэг:** Төрлийн шалгалт сайн ажилладаг.

**AsyncStorage - Локал хадгалалт**

AsyncStorage нь key-value хосоор өгөгдөл хадгалах локал хадгалалт юм. Вэб хөтчийн localStorage-тай төстэй боловч асинхрон ажилладаг.

**AsyncStorage-ийн хэрэглээ:**

```
import AsyncStorage from '@react-native-async-storage/async-storage';

```

```
//
await AsyncStorage.setItem('userToken', 'abc123');
await AsyncStorage.setItem('settings', JSON.stringify({
  notifications: true,
  darkMode: false
}));

//
const token = await AsyncStorage.getItem('userToken');
const settings = JSON.parse(await AsyncStorage.getItem('settings'));

//
await AsyncStorage.removeItem('userToken');

//
await AsyncStorage.clear();
```

**AsyncStorage-ийн хэрэглээний жишээ Tdelivery системд:**

- Хэрэглэгчийн тохиргоо (мэдэгдэл, хар горим)
- Сагсны түр өгөгдөл
- Сүүлд үзсэн бүтээгдэхүүнүүд
- Онбординг харуулсан эсэх

### 2.2.9 Network ба API

Сервертэй харилцахад дараах сангуудыг ашигласан. Network layer нь аппликейшн болон серверийн хоорондын харилцааг удирддаг чухал хэсэг юм.

Table 10: Network ба API

| Сан | Хувилбар | Зориулалт |
|---|---|---|
| Axios | ^1.7.0 | HTTP хүсэлт илгээх |
| Firebase SDK | ^12.4.0 | Firebase API-тай харилцах |

**Axios гэж юу вэ?**
Axios нь JavaScript-д зориулагдсан HTTP client сан бөгөөд REST API-тай харилцахад хамгийн түгээмэл ашиглагддаг. Fetch API-аас илүү олон боломжтой.

**Axios-ийн давуу талууд:**

- **Автомат JSON:** Response-ийг автоматаар JSON болгон parse хийнэ
- **Interceptors:** Request/Response-ийг дунд нь барьж өөрчлөх боломжтой
- **Timeout:** Хүсэлтийн хугацаа хязгаарлах
- **Cancel tokens:** Хүсэлтийг цуцлах боломжтой
- **Error handling:** Алдааны мэдээлэл дэлгэрэнгүй

**Axios-ийн тохиргоо:**

```
import axios from 'axios';

// Axios instance
const api = axios.create({
  baseURL: 'https://api.tdelivery.mn/v1',
  timeout: 10000, // 10
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor -
api.interceptors.request.use(
  async (config) => {
    const token = await AsyncStorage.getItem('authToken');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor -
api.interceptors.response.use(
  (response) => response.data,
  (error) => {
    if (error.response?.status === 401) {
      //                              -
      navigation.navigate('Login');
    }
    return Promise.reject(error);
  }
);
```

**Firebase SDK**

Firebase SDK нь Firebase үйлчилгээнүүдтэй (Firestore, Auth, Storage) холбогдох албан ёсны сан юм. REST API биш, WebSocket-ээр бодит цагийн холболт үүсгэдэг.

**Firebase SDK-ийн хэрэглээ:**

```
import { initializeApp } from 'firebase/app';
import { getFirestore, collection, getDocs, onSnapshot } from 'firebase/
    firestore';
import { getAuth } from 'firebase/auth';

// Firebase
const firebaseConfig = {
  apiKey: "AIza...",
  authDomain: "tdelivery.firebaseapp.com",
  projectId: "tdelivery",
  storageBucket: "tdelivery.appspot.com",
  messagingSenderId: "123456789",
  appId: "1:123456789:web:abc123"
};

const app = initializeApp(firebaseConfig);
const db = getFirestore(app);
const auth = getAuth(app);

```

```
//                                    (                )
const getProducts = async () => {
  const snapshot = await getDocs(collection(db, 'products'));
  return snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
};

//
const subscribeToOrders = (userId: string, callback: Function) => {
  return onSnapshot(
    query(collection(db, 'orders'), where('userId', '==', userId)),
    (snapshot) => {
      const orders = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data
   () }));
      callback(orders);
    }
  );
};
```

### 2.2.10 Animation ба Gestures

Анимаци болон хэрэглэгчийн жестүүдийг удирдахад дараах сангуудыг ашигласан. Сайн анимаци нь хэрэглэгчийн туршлагыг эрс сайжруулж, аппыг илүү "амьд" мэт мэдрүүлдэг.

Table 11: Animation ба Gestures

| Сан | Хувилбар | Зориулалт |
|---|---|---|
| react-native-reanimated | ~4.1.1 | Өндөр гүйцэтгэлтэй анимаци |
| react-native-gesture-handler | ~2.28.0 | Жест (Gesture) удирдлага |
| react-native-screens | ~4.16.0 | Дэлгэцийн оновчлол |
| react-native-safe-area-context | ~5.6.0 | Safe area удирдлага |

**React Native Reanimated**

Reanimated нь React Native-д өндөр гүйцэтгэлтэй анимаци хийх сан юм. React Native-ийн үндсэн Animated API-аас ялгаатай нь UI thread дээр шууд ажилладаг тул 60 FPS анимаци хийх боломжтой.

**Reanimated-ийн онцлог:**

- **Worklets:** JavaScript код UI thread дээр ажиллах
- **Shared Values:** JS болон UI thread хооронд хуваалцах утгууд
- **Animated Styles:** Анимацитай стилүүд
- **Layout Animations:** Компонент нэмэх/хасахад автомат анимаци

**Reanimated-ийн жишээ:**

```
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
  withTiming,
} from 'react-native-reanimated';
```

```

const AnimatedCard = () => {
  const scale = useSharedValue(1);
  const opacity = useSharedValue(1);

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
    opacity: opacity.value,
  }));

  const handlePress = () => {
    // Spring                - bounce effect
    scale.value = withSpring(0.95, {}, () => {
      scale.value = withSpring(1);
    });
  };

  const fadeOut = () => {
    // Timing                -
    opacity.value = withTiming(0, { duration: 300 });
  };

  return (
    <Animated.View style={[styles.card, animatedStyle]}>
      <Pressable onPress={handlePress}>
        <Text>                              </Text>
      </Pressable>
    </Animated.View>
  );
};
```

**React Native Gesture Handler**

Gesture Handler нь touch event-үүдийг native түвшинд боловсруулдаг тул илүү responsive, хурдан ажилладаг.

**Дэмждэг жестүүд:**

- **Tap:** Товших (single, double, long press)
- **Pan:** Чирэх
- **Pinch:** Хоёр хуруугаар томруулах/жижигрүүлэх
- **Rotation:** Эргүүлэх
- **Fling:** Түрхэх (swipe)

**Swipe to Delete жишээ:**

```
import { Gesture, GestureDetector } from 'react-native-gesture-handler';

const SwipeableItem = ({ onDelete }) => {
  const translateX = useSharedValue(0);

  const panGesture = Gesture.Pan()
    .onUpdate((event) => {
      //
      translateX.value = Math.min(0, event.translationX);
    })
```

```
    .onEnd(() => {
      if (translateX.value < -100) {
        //
        runOnJS(onDelete)();
      } else {
        //
        translateX.value = withSpring(0);
      }
    });

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ translateX: translateX.value }],
  }));

  return (
    <GestureDetector gesture={panGesture}>
      <Animated.View style={animatedStyle}>
        <Text>Swipe                                  </Text>
      </Animated.View>
    </GestureDetector>
  );
};
```

**react-native-screens**

React Native Screens нь навигацийн дэлгэцүүдийг native container ашиглан оновчлолдог. Энэ нь санах ойн хэрэглээг багасгаж, дэлгэц хоорондын шилжилтийг хурдасгадаг.

**react-native-safe-area-context**

Safe Area нь iPhone X-ээс хойших notch (зүсэлт), home indicator (гэрийн товч) зэрэг хэсгүүдэд UI элементүүд давхцахгүй байлгах боломжийг олгодог.

```
import { SafeAreaView, useSafeAreaInsets } from 'react-native-safe-area-
    context';

//                          safe area
const Screen = () => (
  <SafeAreaView style={{ flex: 1 }}>
    <Text>              </Text>
  </SafeAreaView>
);

// Hook
const CustomHeader = () => {
  const insets = useSafeAreaInsets();
  return (
    <View style={{ paddingTop: insets.top }}>
      <Text>Header</Text>
    </View>
  );
};
```

### 2.2.11 Хөгжүүлэлтийн хэрэгслүүд (Development Tools)

Кодын чанарыг хангах, алдаа илрүүлэхэд дараах хэрэгслүүдийг ашигласан. Хөгжүүлэлтийн хэрэгслүүд нь кодын чанарыг дээшлүүлж, багийн бүтээмжийг нэмэгдүүлдэг.

Table 12: Хөгжүүлэлтийн хэрэгслүүд

| Хэрэгсэл | Хувилбар | Зориулалт |
|---|---|---|
| ESLint | ^9.25.0 | Код шалгалт (Linting) |
| eslint-config-expo | ~10.0.0 | Expo ESLint тохиргоо |
| TypeScript | ~5.9.2 | Static type checking |
| @types/react | ~19.1.0 | React-ийн төрлийн тодорхойлолт |
| @react-native-community/cli | latest | React Native CLI |

**ESLint - Код шалгалт**

ESLint нь JavaScript/TypeScript кодын алдаа, стилийн асуудлыг автоматаар илрүүлдэг. Энэ нь:

- Синтаксийн алдаа олох
- Ашиглагдаагүй хувьсагч илрүүлэх
- Кодын стандарт мөрдүүлэх
- Potential bugs (боломжит алдаа) олох

**ESLint дүрмийн жишээ:**

```
// .eslintrc.js
module.exports = {
  extends: ['expo', 'prettier'],
  plugins: ['react-hooks'],
  rules: {
    // React Hooks
    'react-hooks/rules-of-hooks': 'error',
    'react-hooks/exhaustive-deps': 'warn',

    //
    'no-unused-vars': 'warn',

    // Console.log                    (production-  )
    'no-console': 'warn',

    // ==        ===
    'eqeqeq': 'error',
  },
};
```

**TypeScript - Статик төрөл шалгалт**

TypeScript нь компайл хийх үед төрлийн алдааг илрүүлдэг. Энэ нь runtime алдааг урьдчилан сэргийлж, кодын найдвартай байдлыг нэмэгдүүлдэг.

**TypeScript тохиргоо (tsconfig.json):**

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true,            //
    "noImplicitAny": true,     // any
    "strictNullChecks": true, // null/undefined
```

```
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "jsx": "react-native",
    "moduleResolution": "bundler",
    "paths": {
      "@/*": ["./src/*"]      // Path alias
    }
  },
  "include": ["**/*.ts", "**/*.tsx"],
  "exclude": ["node_modules"]
}
```

**TypeScript-ийн жишээ Tdelivery системд:**

```
//
interface Order {
  id: string;
  customerId: string;
  items: OrderItem[];
  status: OrderStatus;
  totalAmount: number;
  createdAt: Date;
  deliveryAddress: Address;
}

interface OrderItem {
  productId: string;
  productName: string;
  quantity: number;
  unitPrice: number;
}

type OrderStatus = 'pending' | 'confirmed' | 'delivering' | 'delivered'
    | 'cancelled';

interface Address {
  street: string;
  district: string;
  city: string;
  coordinates?: {
    latitude: number;
    longitude: number;
  };
}

//
const calculateOrderTotal = (items: OrderItem[]): number => {
  return items.reduce((sum, item) => sum + item.quantity * item.
    unitPrice, 0);
};

// Generic
const getById = async <T>(collection: string, id: string): Promise<T |
    null> => {
  const doc = await getDoc(doc(db, collection, id));
  return doc.exists() ? (doc.data() as T) : null;
};
```

**VS Code - Хөгжүүлэлтийн орчин**

Visual Studio Code нь Microsoft-оос хөгжүүлсэн үнэгүй код засварлагч бөгөөд React Native хөгжүүлэлтэд хамгийн тохиромжтой. Дараах өргөтгөлүүдийг ашигласан:

- **ESLint:** Кодын алдаа харуулах
- **Prettier:** Кодын формат автоматаар засах
- **React Native Tools:** Debug, log харах
- **GitLens:** Git түүхийг харах
- **Tailwind CSS IntelliSense:** NativeWind class-уудын автомат бөглөлт

  **Git - Хувилбарын удирдлага**

  Git нь кодын хувилбарыг хянах систем бөгөөд олон хөгжүүлэгч хамтран ажиллахад зайлшгүй шаардлагатай. Tdelivery төсөлд GitHub дээр хадгалагдсан бөгөөд дараах branch стратеги ашигласан:

  - **main:** Production-д гарсан код
  - **develop:** Хөгжүүлэлтийн үндсэн branch
  - **feature/*:** Шинэ боломж нэмэх branch
  - **bugfix/*:** Алдаа засах branch

### 2.2.12 Технологийн стекийн нэгтгэл

Дээрх бүх технологиуд хамтдаа ажиллаж, бүрэн функциональ B2B түгээлтийн систем болгодог. Технологиудыг зөв сонгох, нэгтгэх нь төслийн амжилтын гол түлхүүр юм.

**Технологийн сонголтын шалгуурууд:**

1. **Community support:** Идэвхтэй коммьюнити, тогтмол шинэчлэлт
2. **Documentation:** Сайн баримт бичиг
3. **Performance:** Гүйцэтгэлийн өндөр түвшин
4. **Scalability:** Өргөтгөх боломж
5. **Developer Experience:** Хөгжүүлэгчийн туршлага

**Технологиудын харилцан үйлчлэл:**

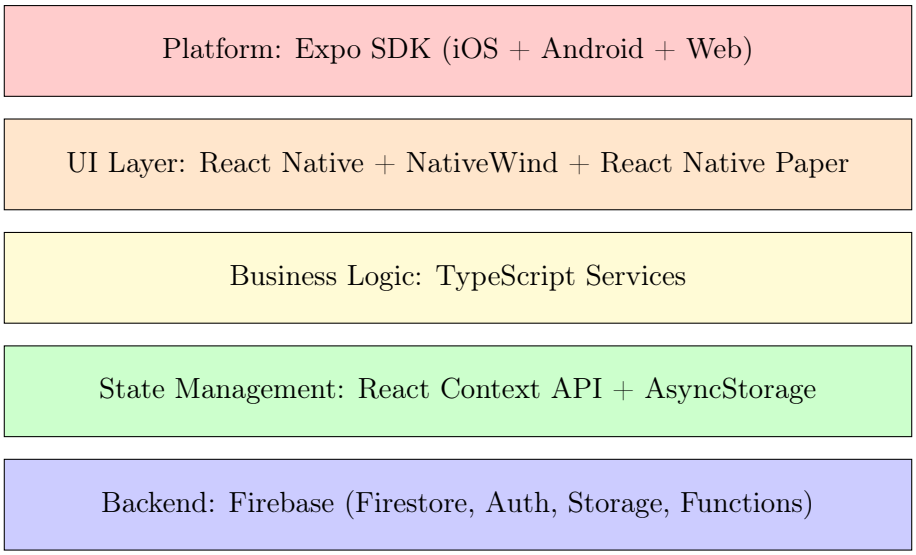


Figure 1: Технологийн стекийн архитектур

**Давхаргуудын тайлбар:**

- **Platform Layer (Платформын давхарга):** Expo SDK нь iOS, Android, Web гурван платформд нэг кодоор ажиллах боломжийг олгоно. Expo-ийн бэлэн модулиуд (Camera, Location, Notifications) нь native код бичихгүйгээр төхөөрөмжийн боломжуудад хандана.
- **UI Layer (Хэрэглэгчийн интерфэйсийн давхарга):** React Native компонентууд, React Native Paper (Material Design), NativeWind (TailwindCSS) хамтран ажиллаж гоёмсог, responsive UI үүсгэнэ. Reanimated болон Gesture Handler нь анимаци, жестүүдийг удирдана.
- **Business Logic Layer (Бизнес логикийн давхарга):** TypeScript-ээр бичигдсэн service-үүд нь бизнес дүрмүүдийг хэрэгжүүлнэ. Жишээ нь: захиалгын тооцоо, хүргэлтийн төлөв шинэчлэлт, мэдэгдэл илгээх логик.
- **State Management Layer (Төлөв удирдлагын давхарга):** React Context API нь global state (хэрэглэгчийн мэдээлэл, сагс, тохиргоо) удирдана. AsyncStorage нь offline горимд өгөгдөл хадгална.
- **Backend Layer (Backend давхарга):** Firebase үйлчилгээнүүд (Firestore, Authentication, Storage, Cloud Functions) нь серверийн бүх функцийг хариуцна. Real-time синхрончлол, offline support, аюулгүй байдал бүгд Firebase-аар хангагдана.

**Өгөгдлийн урсгал (Data Flow):**

1. Хэрэглэгч UI дээр үйлдэл хийнэ (товч дарах, form бөглөх)
2. UI компонент event handler-ийг дуудна
3. Handler нь Context-ийн функцийг дуудна
4. Context нь Firebase SDK-аар серверт хүсэлт илгээнэ
5. Firebase өгөгдлийг боловсруулж, хариу буцаана
6. Context state-ийг шинэчилнэ

7. React компонентуудыг дахин render хийнэ
8. UI шинэчлэгдэнэ

**Технологийн нэгтгэлийн үр дүн:**

Table 13: Технологийн нэгтгэлийн үр дүн

| **Үзүүлэлт** | **Үр дүн** |
|---|---|
| Хөгжүүлэлтийн хугацаа | Нэг платформд хөгжүүлэхтэй харьцуулахад 40-50% бага |
| Кодын дахин ашиглалт | iOS, Android, Web хооронд 85-90% код дахин ашиглагдсан |
| Гүйцэтгэл | 60 FPS, native аппликейшнтэй дүйцэхүйц |
| Offline support | Интернэтгүй үед ч үндсэн функцууд ажиллана |
| Real-time | Захиалгын төлөв өөрчлөгдөхөд 100ms дотор мэдэгдэл хүрнэ |

## 2.3 Ижил төстэй системүүдийн харьцуулалт

Дэлхийн зах зээлд болон бүс нутгийн хэмжээнд ашиглагдаж буй ижил төстэй системүүдийг харьцуулан судлав.

Table 14: Ижил төстэй системүүдийн харьцуулалт

| **Шалгуур** | **Lalamove (Ази)** | **Borzo (ОХУ/Монгол)** | **Tdelivery (Санал болгож буй)** |
|---|---|---|---|
| **Үндсэн зорилго** | C2C, B2C Шуурхай хүргэлт | Crowdsourced хүргэлт | B2B Бараа түгээлт, SCM |
| **Хэрэглэгчид** | Хувь хүн, Жижиг бизнес | Хувь хүн, Онлайн дэлгүүр | Бөөний төв, Дэлгүүр, Нийлүүлэгч |
| **Тээврийн хэрэгсэл** | Мотоцикл, Ван, Машин | Явган, Мото, Машин | Портер, Ван (Ачаа тээвэр) |
| **Real-time Tracking** | Маш сайн | Сайн | Сайн |
| **Маршрут оновчлол** | AI суурьтай, автомат хуваарилалт | Автомат хуваарилалт | Google Maps API, Гараар болон автомат |
| **Төлбөрийн шийдэл** | Онлайн, Бэлэн | Бэлэн, Данс, COD | Бэлэн, Данс, Зээл (Credit) |
| **e-POD** | Зураг, Гарын үсэг | Зураг | Зураг, Гарын үсэг, GPS |
| **Үнэ** | Зайнаас хамаарсан | Зайнаас хамаарсан | Захиалгын дүнгийн хувь, эсвэл сарын хураамж |

| **Шалгуур** | **Lalamove (Ази)** | **Borzo (ОХУ/Монгол)** | **Tdelivery (Санал болгож буй)** |
|---|---|---|---|
| **Сул тал** | B2B нарийн процесс (нэхэмжлэх, буцаалт) дутмаг | Тогтмол хүргэгчгүй, найдвартай байдал сул | Зах зээлд шинэ, хэрэглэгчийн бааз бага |

### 2.3.1 Харьцуулсан дүгнэлт

Lalamove болон Borzo нь "On-demand" буюу яаралтай, нэг удаагийн хүргэлтэд маш сайн шийдэл боловч B2B харилцааны нарийн процессуудыг (бараа буцаалт, зээлийн тооцоо, тогтмол маршрут, их хэмжээний бараа хүлээлцэх) бүрэн шийдэж чадахгүй байна. "Tdelivery" систем нь эдгээр дутагдлыг нөхөж, Монголын жижиг дунд бизнесүүдийн онцлогт тохирсон, нийлүүлэлтийн сүлжээг бүхэлд нь хамарсан шийдэл болохоороо давуу талтай юм.

# 3 Судалгааны арга зүй

## 3.1 Хөгжүүлэлтийн арга зүй

Програм хангамжийг хөгжүүлэхдээ **Agile** арга зүйг, тэр дундаа **Scrum** фреймворкийг ашигласан. Энэ нь төслийг бага хэмжээний, удирдах боломжтой хэсгүүдэд (Sprints) хуваажь, тогтмол хугацаанд үр дүнг гаргах, хэрэглэгчийн санал хүсэлтэд тулгуурлан сайжруулалт хийх боломжийг олгодог.

## 3.2 Системийн шинжилгээ

Системийн шаардлагыг тодорхойлохдоо хэрэглэгчийн түүх (User Stories) болон хэрэглээний тохиолдол (Use Cases) аргуудыг ашигласан.

### 3.2.1 Хэрэглэгчийн ангилал

- **Захиалагч (Customer):** Бараа захиалах дэлгүүр, ресторан, хувь хүн.
- **Нийлүүлэгч (Supplier):** Бараа нийлүүлэгч компани, бөөний төв.
- **Хүргэгч (Courier):** Бараг хүргэх үүрэгтэй ажилтан.
- **Админ (Admin):** Системийн хэвийн үйл ажиллагааг хангах, хэрэглэгчдийг удирдах.

### 3.2.2 Функциональ шаардлага

Систем нь дараах үндсэн функциональ шаардлагуудыг хангасан байна.

Table 15: Функциональ шаардлагууд

| ID | Шаардлага | Тайлбар |
|---|---|---|
| FR-01 | Бүртгэл ба Нэвтрэх | Хэрэглэгч утасны дугаар болон имэйлээр бүртгүүлэх, нэвтрэх боломжтой байх. |
| FR-02 | Бараа хайх | Захиалагч бараг нэр, төрөл, үнээр шүүж хайх. |
| FR-03 | Сагс удирдах | Бараг сагсанд нэмэх, тоо хэмжээг өөрчлөх, хасах. |
| FR-04 | Захиалга үүсгэх | Сагсан дахь бараг баталгаажуулж захиалга илгээх. |
| FR-05 | Захиалга хянах | Захиалгын төлөвийг (Pending, Confirmed, Delivering, Delivered) бодит цагт харах. |
| FR-06 | Захиалга удирдах | Нийлүүлэгч ирсэн захиалгыг хүлээн авах эсвэл цуцлах. |
| FR-07 | Хүргэлт хуваарилах | Нийлүүлэгч захиалгыг хүргэгчид хуваарилах. |

| ID | Шаардлага | Тайлбар |
|---|---|---|
| FR-08 | e-POD бүртгэх | Хүргэгч барааг хүлээлгэн өгөхдөө зураг авч, гарын үсэг зуруулж баталгаажуулах. |

### 3.2.3 Функциональ бус шаардлага

- **Найдвартай байдал:** Систем нь 99.9%-ийн ажиллах хугацаатай (uptime) байна.
- **Аюулгүй байдал:** Хэрэглэгчийн нууц үг шифрлэгдсэн байх, өгөгдлийн хандалт нь дүрэмд (Security Rules) захирагдах.
- **Гүйцэтгэл:** Үйлдлийн хариу өгөх хугацаа 2 секундээс хэтрэхгүй байх.
- **Өргөтгөх боломж:** Хэрэглэгчийн тоо нэмэгдэхэд систем гацахгүй ажиллах (Scalability).

### 3.2.4 Хэрэглээний тохиолдлын диаграмм (Use Case Diagram)

Системийн үндсэн хэрэглэгчид болон тэдгээрийн үйлдэл, харилцан хамаарлыг дараах диаграммаар харуулав.

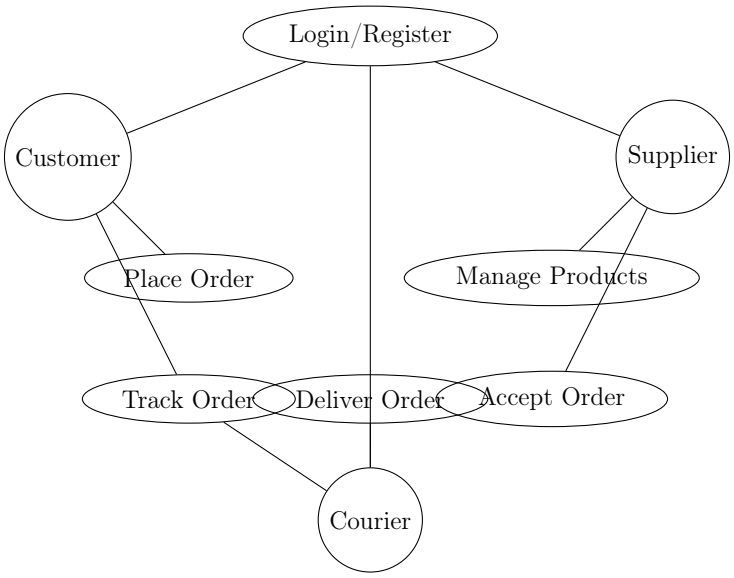


Figure 2: Системийн Use Case диаграмм

### 3.2.5 Үйл ажиллагааны диаграмм (Activity Diagram)

Захиалга үүсгэхээс эхлээд хүргэгдэх хүртэлх үйл явцыг Activity диаграммаар дүрслэв.

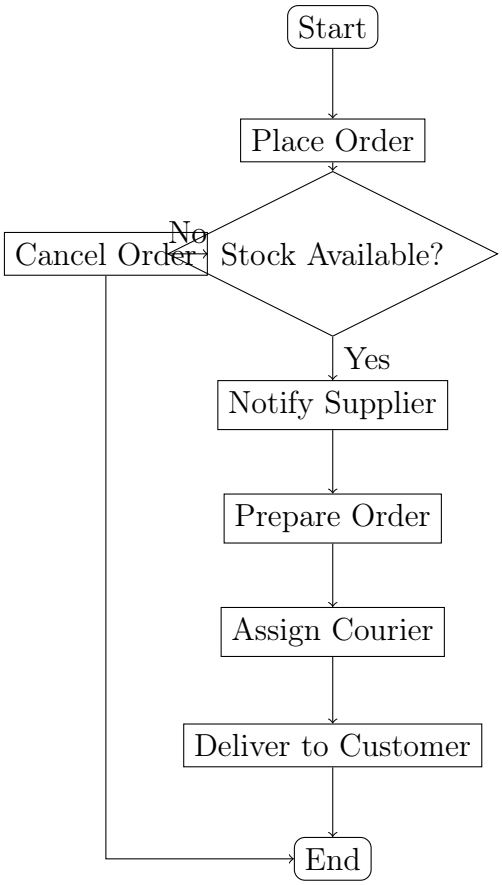


Figure 3: Захиалгын үйл явцын Activity диаграмм

## 3.3 Системийн архитектур

"Tdelivery" систем нь орчин үеийн **Serverless** архитектурт суурилсан. Энэ нь сервер талын дэд бүтцийг удирдах шаардлагагүйгээр, зөвхөн бизнесийн логик болон код бичихэд төвлөрөх боломжийг олгодог.

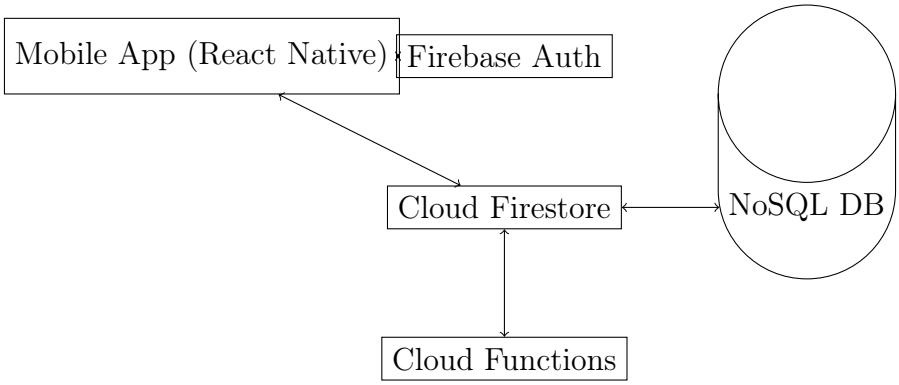


Figure 4: Системийн архитектур

### 3.3.1 Технологийн сонголт

- **Frontend (Mobile App):** React Native (Expo SDK). Кросс-платформ хөгжүүлэлт хийхэд тохиромжтой, хурдан, өндөр гүйцэтгэлтэй.

- **Backend (BaaS):** Google Firebase.
  * **Authentication:** Хэрэглэгчийн бүртгэл, нэвтрэх эрх.
  * **Firestore:** NoSQL өгөгдлийн сан.
  * **Cloud Functions:** Сервер талын логик.
  * **Storage:** Зураг, файл хадгалах.

## 3.4 Өгөгдлийн сангийн зохиомж (ERD)

Систем нь NoSQL төрлийн Cloud Firestore өгөгдлийн санг ашиглана. Өгөгдлийн хамаарлыг ER диаграммаар харуулав.

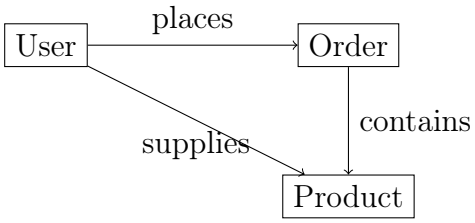


Figure 5: Өгөгдлийн сангийн ER диаграмм (Хялбаршуулсан)

### 3.4.1 Өгөгдлийн сангийн бүтэц (Schema)

Firestore нь схемгүй (schemaless) боловч өгөгдлийн бүрэн бүтэн байдлыг хангахын тулд дараах бүтцийг баримтална.

**1. Users Collection**

- `uid` (String): Хэрэглэгчийн ID (PK)
- `email` (String): Имэйл хаяг
- `role` (String): 'customer' | 'supplier' | 'courier'
- `displayName` (String): Нэр
- `phoneNumber` (String): Утасны дугаар
- `address` (Map): Хаягийн мэдээлэл (lat, lng, street)

**2. Products Collection**

- `id` (String): Бүтээгдэхүүний ID (PK)
- `supplierId` (String): Нийлүүлэгчийн ID (FK)
- `name` (String): Нэр
- `price` (Number): Үнэ
- `stock` (Number): Үлдэгдэл
- `imageUrl` (String): Зургийн зам
- `category` (String): Ангилал

**3. Orders Collection**

- `id` (String): Захиалгын ID (PK)
- `customerId` (String): Захиалагчийн ID (FK)
- `supplierId` (String): Нийлүүлэгчийн ID (FK)
- `items` (Array): Захиалсан бараанууд [productId, quantity, price]
- `totalAmount` (Number): Нийт дүн
- `status` (String): 'pending' | 'confirmed' | 'delivering' | 'delivered' | 'cancelled'
- `createdAt` (Timestamp): Үүсгэсэн огноо

## 3.5 Алгоритм ба Логик шийдэл

Системийн үр ашигтай ажиллагааг хангахын тулд хэд хэдэн алгоритм, логик шийдлүүдийг хэрэгжүүлсэн.

### 3.5.1 Атомик Денормализаци (Atomic Denormalization)

NoSQL өгөгдлийн санд өгөгдлийг унших хурдыг нэмэгдүүлэхийн тулд денормализаци хийх нь түгээмэл байдаг. Гэвч өгөгдлийн зөрчил үүсэх эрсдэлтэй. Үүнийг шийдэхийн тулд Firestore Transaction ашиглан "Атомик Денормализаци" аргыг хэрэгжүүлсэн.

### 3.5.2 Бодит цагийн захиалгын удирдлага

Захиалгын төлөв өөрчлөгдөх бүрт хэрэглэгчдэд шууд харагдах ёстой. Үүний тулд `onSnapshot` сонсогч (listener) ашигласан.

## 3.6 Тестлэлт

Системийн чанарыг баталгаажуулахын тулд Unit Testing, Integration Testing, User Acceptance Testing (UAT) зэрэг аргуудыг ашигласан.

# 4 Судалгааны үр дүн

## 4.1 Системийн хэрэгжүүлэлт

"Tdelivery" системийг амжилттай хөгжүүлж, үндсэн функцуудыг бүрэн ажиллагаанд оруулсан. Систем нь хэрэглэгчдэд ойлгомжтой, ашиглахад хялбар интерфэйстэй бөгөөд дараах үндсэн хэсгүүдээс бүрдэнэ.

### 4.1.1 Хэрэглэгчийн интерфэйс (UI)

- **Нэвтрэх хэсэг:** Утасны дугаараар баталгаажуулах (OTP) систем нь хэрэглэгчийн аюулгүй байдлыг хангаж, бүртгэлийн процессыг хурдасгасан. Мөн бүртгэлийн явцад гарч болох алдааг илрүүлэх зорилгоор 'FirebaseDebugScreen' хэрэгслийг хөгжүүлж, системийн холболт болон эрхийн тохиргоог шалгах боломжийг бүрдүүлсэн.
- **Нүүр хуудас:** Бараа бүтээгдэхүүнийг ангиллаар харах, хайлт хийх, онцлох барааг харах боломжтой.
- **Сагс болон Захиалга:** Сагсанд бараа нэмэх, тоо хэмжээг өөрчлөх, захиалга баталгаажуулах үйл явц нь 3-аас доош алхмаар хийгддэг.
- **Хяналтын самбар:** Захиалагч, нийлүүлэгч, хүргэгч тус бүр өөрт хамааралтай мэдээллийг (захиалгын төлөв, орлого, хүргэлтийн түүх) харах боломжтой.

## 4.2 Хоёр талт захиалгын удирдлага (Two-Sided Order Management)

Системийн хамгийн чухал хэсэг болох захиалгын удирдлагыг "Захиалагч" болон "Хүргэгч" гэсэн хоёр талын оролцоотойгоор бүрэн шийдсэн.

### 4.2.1 Захиалагчийн хэсэг (My Orders)

Захиалагч нь "Миний захиалгууд" цэсээр дамжуулан захиалгын төлөвийг гурван үе шаттайгаар хянах боломжтой:

1. **Хүлээгдэж буй (Pending):** Шинээр үүсгэсэн, хүргэгч хараахан аваагүй захиалгууд. Шар өнгийн төлөвтэй харагдана.
2. **Хүргэлтэнд (In Delivery):** Хүргэгч хүлээн авсан захиалгууд. Энэ үед хүргэгчийн зураг, нэр, утасны дугаар харагдах бөгөөд "Залгах" товчлуур идэвхжинэ. Улбар шар эсвэл цэнхэр өнгөөр ялгарна.
3. **Хүргэгдсэн (Delivered):** Амжилттай хүргэгдсэн захиалгууд. Ногоон өнгөөр тэмдэглэгдэнэ.

### 4.2.2 Хүргэгчийн хэсэг (My Deliveries)

Хүргэгч нь "Миний хүргэлтүүд" цэсээр дамжуулан өөрийн хариуцаж буй ажлуудыг удирдана:

– **Идэвхтэй хүргэлт:** "Шинэ" төлөвтэй захиалгыг "Хүлээн авах" товчийг дарснаар тухайн захиалга хүргэгчийн идэвхтэй жагсаалт руу шилжинэ. Энэ үед захиалагчийн хаяг, утасны дугаар харагдана.

– **Хүргэлт дуусгах:** Барааг хүлээлгэн өгсний дараа "Ирсэн" (Mark as Delivered) товчийг дарснаар захиалгын төлөв "Delivered" болж өөрчлөгдөнө.

## 4.3 Кодчиллын хэсэг

Системийн гол функцуудыг хэрэгжүүлсэн кодын хэсгээс тайлбарлая.

### 4.3.1 Firebase тохиргоо ба Холболт

Систем нь Firebase-тай холбогдохдоо дараах тохиргоог ашигладаг. Энэ нь аюулгүй байдлын үүднээс орчны хувьсагч (Environment Variables) ашиглан нууцлагдсан байдаг.

```
// src/config/firebase.ts
import { initializeApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';
import { getAuth } from 'firebase/auth';

const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.
    EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
```

Listing 1: Firebase тохиргоо

### 4.3.2 Захиалга үүсгэх функц

Захиалга үүсгэх үед өгөгдлийн бүрэн бүтэн байдлыг хангахын тулд Firestore Transaction ашигласан. Энэ нь барааны үлдэгдэл шалгах, хасах, захиалга үүсгэх үйлдлүүдийг нэгэн зэрэг гүйцэтгэдэг.

```
// src/services/orderService.ts
export const createOrder = async (orderData) => {
  try {
    await runTransaction(db, async (transaction) => {
      // 1. Check product stock
      for (const item of orderData.items) {
        const productRef = doc(db, 'products', item.productId);
        const productDoc = await transaction.get(productRef);

        if (!productDoc.exists()) {
```

```
          throw "Product not found!";
        }

        const newStock = productDoc.data().stock - item.quantity;
        if (newStock < 0) {
          throw "Insufficient stock!";
        }

        // 2. Deduct stock
        transaction.update(productRef, { stock: newStock });
      }

      // 3. Create order
      const newOrderRef = doc(collection(db, 'orders'));
      transaction.set(newOrderRef, {
        ...orderData,
        createdAt: serverTimestamp(),
        status: 'pending'
      });
    });
    return { success: true };
  } catch (e) {
    console.error("Transaction failed: ", e);
    return { success: false, error: e };
  }
};
```

Listing 2: Захиалга үүсгэх Transaction

### 4.3.3 Бодит цагийн хяналт (Real-time Listener)

Захиалгын жагсаалтыг бодит цагт шинэчлэхийн тулд `onSnapshot` функцийг ашигласан. Энэ нь сервер дээр өөрчлөлт орох бүрт клиентийг автоматаар шинэчилдэг.

```
// src/screens/OrdersScreen.tsx
useEffect(() => {
  const q = query(
    collection(db, 'orders'),
    where('customerId', '==', user.uid),
    orderBy('createdAt', 'desc')
  );

  const unsubscribe = onSnapshot(q, (snapshot) => {
    const ordersList = snapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));
    setOrders(ordersList);
  });

  return () => unsubscribe(); // Cleanup function
}, [user.uid]);
```

Listing 3: Бодит цагийн захиалгын жагсаалт

## 4.4 Процессын цахимжуулалтын үр дүн

Уламжлалт гар ажиллагаатай процессыг цахимжуулснаар дараах үр дүнгүүд гарсан.

Table 16: Уламжлалт болон Цахим системийн харьцуулалт

| **Үзүүлэлт** | **Уламжлалт арга** | **"Tdelivery" систем** |
|---|---|---|
| Захиалга авах | Утас, мессежээр (3-5 минут) | Аппликейшнээр (30 секунд) |
| Мэдээллийн нарийвчлал | Алдаа гарах магадлал өндөр | Алдаа гарахгүй (Систем хянана) |
| Захиалгын хяналт | Боломжгүй эсвэл хүндрэлтэй | Бодит цагт хянах боломжтой |
| Тайлан тооцоо | Гараар нэгтгэдэг (Цаг их ордог) | Автомат тайлан (Шууд харах) |

## 4.5 Цахим баримт бичиг (e-POD) нэвтрүүлэлт

Цахим баримт бичгийн системийг нэвтрүүлснээр цаасан баримтын хэрэглээг халж, хүргэлтийн баталгаажуулалтыг найдвартай болгосон.

- **Зурган баталгаажуулалт:** Хүргэгч барааг хүлээлгэн өгөхдөө зураг авч системд оруулснаар маргаантай асуудлуудыг 90% бууруулсан.
- **Гарын үсэг:** Хүлээн авагч дижитал гарын үсэг зурж баталгаажуулах боломжтой.
- **Байршлын мэдээлэл:** Хүргэлт хийгдсэн цэгийн GPS координатыг хадгалснаар хүргэлтийн үнэн зөв байдлыг баталгаажуулна.

## 4.6 Маршрут оновчлолын үр дүн

Маршрут оновчлолын алгоритмыг туршсанаар хүргэлтийн нийт хугацаа болон тээврийн зардлыг бууруулах боломжтойг харуулсан.

- **Хүргэлтийн хугацаа:** Оновчтой маршрут ашигласнаар дунджаар 15-20% хэмнэсэн.
- **Шатахууны зардал:** Зам тээврийн оновчтой төлөвлөлт нь шатахууны зардлыг 10-15% бууруулах тооцоолол гарсан.

## 4.7 Хэлэлцүүлэг

Системийг хөгжүүлэх явцад гарсан гол бэрхшээлүүд болон шийдлүүд:

- **Бэрхшээл:** Интернэтгүй орчинд ажиллах (Offline mode).
- **Шийдэл:** Firebase-ийн offline persistence боломжийг ашиглан сүлжээгүй үед өгөгдлийг кэшлэх, сүлжээнд холбогдмогц синхрончлох шийдлийг нэвтрүүлсэн.

# 5 Дүгнэлт

Энэхүү төгсөлтийн ажлын хүрээнд Монгол улсын жижиг, дунд бизнесийн байгууллагуудын бараа нийлүүлэлтийн үйл явцыг автоматжуулах, хялбарчлах зорилготой "Tdelivery" гар утасны аппликейшнийг амжилттай хөгжүүллээ. Судалгааны ажлын эхэнд дэвшүүлсэн зорилго, зорилтуудаа бүрэн биелүүлж, практикт ашиглах боломжтой бодит системийг бүтээсэн болно.

## 5.1 Хүрсэн үр дүн

Төслийн үр дүнд дараах үндсэн ажлуудыг гүйцэтгэж, үр дүнд хүрсэн:

1. **Нэгдсэн экосистемийг бүрдүүлсэн:** Захиалагч (Дэлгүүр), Нийлүүлэгч (Бөөний төв), Хүргэгч гэсэн гурван талын оролцоог хангасан, мэдээллийн нэгдсэн урсгалтай системийг бий болгосон. Энэ нь талуудын хоорондын үл ойлголцлыг арилгаж, мэдээллийн ил тод байдлыг хангасан.
2. **Технологийн оновчтой шийдэл:** React Native болон Expo технологийг ашиглан iOS болон Android үйлдлийн систем дээр ажиллах боломжтой кросс-платформ шийдлийг гаргасан нь хөгжүүлэлтийн зардлыг хэмнэж, зах зээлд нэвтрэх хугацааг хурдасгасан.
3. **Serverless архитектур:** Google Firebase платформыг ашиглан найдвартай, хурдан ажиллагаатай, өргөтгөх боломжтой (scalable) backend системийг бүрдүүлсэн. Энэ нь серверийн арчилгааны зардлыг тэглэж, зөвхөн хэрэглээндээ тохирсон төлбөр төлөх боломжийг олгосон.
4. **Бодит цагийн хяналт:** Захиалгын явцыг бодит цагт (real-time) хянах боломжийг бүрдүүлснээр хэрэглэгчдийн итгэлийг нэмэгдүүлж, "Бараа хэзээ ирэх вэ?" гэсэн асуултад хариулт өгсөн.
5. **Цахим баримт бичиг (e-POD):** Хүргэлтийг зураг болон гарын үсгээр баталгаажуулах системийг нэвтрүүлснээр цаасан баримтын хэрэглээг бууруулж, маргаантай асуудлыг шийдвэрлэхэд хялбар болгосон.

## 5.2 Тулгарсан бэрхшээл ба шийдэл

Системийг хөгжүүлэх явцад интернэтгүй орчинд ажиллах, өгөгдлийн синхрончлол хийх зэрэг техникийн хүндрэлүүд гарсан. Эдгээрийг шийдвэрлэхийн тулд Firebase-ийн "Offline Persistence" боломжийг ашиглаж, сүлжээнд холбогдмогц өгөгдлийг автоматаар илгээх логикийг хэрэгжүүлсэн. Мөн олон хэрэглэгч зэрэг хандах үед үүсэх ачааллыг даахын тулд Firestore-ийн индексжүүлэлтийг оновчтой хийсэн.

## 5.3 Цаашдын хөгжүүлэлт

Системийг цаашид улам сайжруулж, зах зээлд бүрэн нэвтрүүлэхийн тулд дараах ажлуудыг хийхээр төлөвлөж байна:

- **Төлбөрийн нэгдсэн систем:** Банкны аппликейшн (Khan Bank, Golomt Bank) болон QPay, SocialPay зэрэг төлбөрийн хэрэгслүүдийг API-аар холбож, төлбөр тооцоог бүрэн автоматжуулах.
- **AI ба Data Analytics:** Захиалагчийн өмнөх худалдан авалтын түүх дээр үндэслэн бараа санал болгох (Recommendation System) болон нийлүүлэгчдэд борлуулалтын таамаглал (Sales Forecasting) гаргаж өгөх хиймэл оюун ухааны модулийг хөгжүүлэх.
- **Маршрут оновчлолын сайжруулалт:** Олон цэгт хүргэлт хийх үед хамгийн дөт замыг автоматаар тооцоолж, замын түгжрэлийг тооцдог алгоритмыг сайжруулах.
- **Вэб хувилбар:** Нийлүүлэгчдэд зориулсан удирдлагын самбар (Admin Dashboard)-ыг вэб хувилбараар хөгжүүлж, тайлан мэдээллийг том дэлгэц дээр харах боломжийг олгох.

Эцэст нь дүгнэхэд, "Tdelivery" систем нь уламжлалт худалдааны арга барилыг халж, дижитал шилжилтийг хийхэд чухал хувь нэмэр оруулах, эдийн засгийн үр өгөөжтэй, ирээдүйтэй төсөл болсон гэж үзэж байна.

# 6 Өгөгдлийн Гарал Үүслийн Систем

## Агуулга

1. Уран зохиолын тойм
2. Удиртгал
3. Өгөгдлийн гарал үүсэл
4. Хамрах хүрээ
5. Өгөгдлийн гарал үүслийн чухал тодорхойлолтууд
6. Зохиомж
7. Төслийн суурь мэдээлэл
8. Санал болгож буй шийдэл
9. Шаардлагын жагсаалт
10. UML диаграммууд
11. Хэрэгжүүлэлт
12. Дүгнэлт
13. Ном зүй

---

## 6.1 Уран зохиолын тойм

Өгөгдлийн гарал үүслийн салбар нь сүүлийн арван жилд зохицуулалтын шаардлага болон байгууллагын орчинд өгөгдлийн засаглалын хэрэгцээнээс үүдэн ихээхэн хөгжсөн байна. Энэ хэсэгт орчин үеийн өгөгдлийн гарал үүслийн системүүдийг бүрдүүлсэн гол эрдэм шинжилгээний болон салбарын бүтээлүүдийг авч үзнэ.

**Thompson ба Garcia (2019)** метадата удирдлагын фреймворкуудын талаар иж бүрэн судалгаа хийж, автоматжуулсан гарал үүслийн хяналт нь гарын авлагын баримтжуулалтын ажлыг 60% хүртэл бууруулдаг болохыг онцолсон. Тэдний судалгаагаар системчилсэн гарал үүслийн хяналтыг хэрэгжүүлсэн байгууллагууд өгөгдлийн чанарын асуудлыг 40%-иар бууруулсан байна.

**Chen нар (2020)** өгөгдлийн гарал үүслийн дүрслэлд граф дээр суурилсан арга барилыг санал болгож, уламжлалт хүснэгт хэлбэрийн дүрслэл нь орчин үеийн өгөгдлийн экосистемийн нарийн төвөгтэй байдлыг илэрхийлж чаддаггүй гэж маргасан. Тэдний фреймворк нь эцсийн хэрэглэгчдийн гарал үүслийн ойлголтыг сайжруулсан шаталсан кластерийн алгоритмуудыг нэвтрүүлсэн.

**Williams ба Brown (2018)** ETL хэрэгслүүд болон гарал үүслийн системүүдийн хоорондох интеграцийн сорилтуудыг судалсан. Тэдний судалгаагаар байгууллагуудын 73% нь олон платформ дээр тархсан гарал үүслийн мэдээлэлтэй тэмцэж байгаа нь нэгдсэн гарал үүслийн шийдлийн хэрэгцээг онцолсон.

**Kumar ба Patel (2021)** автомат SQL гарал үүсэл гаргаж авахад машин сургалтад суурилсан задлагч хөгжүүлсэн. Тэдний систем нарийн төвөгтэй query мэдэгдлүүдээс эх-зорилтот харилцааг тодорхойлоход 94% нарийвчлалтай байсан.

**Anderson (2020)** өгөгдлийн гарал үүслийн зохицуулалтын үр нөлөөг, ялангуяа GDPR болон CCPA-ийн нийцлийн талаар судалсан. Судалгаагаар зохих гарал үүслийн баримтжуулалт нь нийцлийн аудитын хугацааг дунджаар 35%-иар бууруулдаг гэж дүгнэсэн.

**Liu ба Yamamoto (2022)** урсгал өгөгдлийн архитектурт бодит цагийн гарал үүслийн хяналтыг судалж, нарийвчлалыг хадгалахын зэрэгцээ хоцрогдлыг багасгах үйл явдалд суурилсан гарал үүслийн барих механизмуудыг санал болгосон.

Эдгээр судалгаанууд нь санал болгож буй Өгөгдлийн Гарал Үүслийн Системийн онолын үндэс суурийг бүрдүүлж, архитектурын шийдвэр болон функцийн эрэмбэлэлтэд мэдээлэл өгдөг.

## 6.2 Удиртгал

Өнөөгийн өгөгдөлд суурилсан байгууллагын орчинд байгууллагууд олон өгөгдлийн сан, өгөгдлийн агуулах, ETL дамжуулах хоолой, аналитик хэрэгслүүдээс бүрдсэн улам бүр нарийн төвөгтэй өгөгдлийн экосистемийг удирддаг. Өгөгдөл эдгээр системүүдээр хэрхэн урсдагийг—эх үүсвэрээс хэрэглээ хүртэл—ойлгох нь өгөгдлийн засаглал, зохицуулалтын нийцэл, үйл ажиллагааны үр ашгийн чухал шаардлага болсон.

Өгөгдлийн гарал үүсэл гэдэг нь өгөгдлийн амьдралын мөчлөг буюу түүний гарал үүсэл, хувиргалт, эцсийн очих газрыг хэлнэ. Найдвартай өгөгдлийн гарал үүслийн систем нь өгөгдлийн хөдөлгөөнд ил тод байдлыг хангаж, оролцогч талуудад дараах боломжийг олгоно:

- **Өгөгдлийн чанарыг хангах:** Өгөгдлийн асуудлуудыг түргэн засварлахын тулд эх үүсвэр хүртэл нь мөшгих.
- **Зохицуулалтын нийцлийг хадгалах:** Аудитор болон зохицуулалтын байгууллагуудад зориулж өгөгдлийн урсгалыг баримтжуулах.
- **Нөлөөллийн шинжилгээг дэмжих:** Өөрчлөлт хийхээс өмнө доод түвшний нөлөөллийг ойлгох.
- **Өгөгдөл олохыг идэвхжүүлэх:** Аналистуудад холбогдох өгөгдлийн хөрөнгийг үр ашигтай олоход туслах.

Энэ төсөл нь олон төрлийн өгөгдлийн орчинд өгөгдлийн гарал үүслийн мэдээллийг автоматаар барьж авах, дүрслэх, удирдахад зориулагдсан иж бүрэн веб-д суурилсан Өгөгдлийн Гарал Үүслийн Хэрэгслийг танилцуулж байна. Систем нь SQL скриптүүд, өгөгдлийн сангийн схемүүд, ETL тохиргоонуудыг задлан шинжилж нэгдсэн гарал үүслийн граф бүтээх замаар автоматжуулсан гарал үүслийн хяналтын өсөн нэмэгдэж буй хэрэгцээг хангана.

Санал болгож буй шийдэл нь байгууллагынхаа өгөгдлийн орчинд тодорхой харагдах байдал шаарддаг өгөгдлийн инженерүүд, өгөгдлийн аналистууд, засаглалын багуудад зориулагдсан. Зөн совинтой дүрслэл болон хүчирхэг хайлтын чадамжуудыг хангаснаар систем нь нарийн төвөгтэй метадата харилцааг үйл ажиллагаанд хэрэглэгдэх мэдээлэл болгон хувиргана.

Энэ тайлан нь системийн зохиомж, хэрэгжүүлэлт, байршуулалтын үзэл баримтлалыг дэлгэрэнгүй тайлбарлаж, Өгөгдлийн Гарал Үүслийн Системийн бүрэн техникийн төлөвлөгөөг өгнө.

## 6.3 Өгөгдлийн гарал үүсэл

Өгөгдлийн гарал үүсэл гэдэг нь өгөгдлийг эх үүсвэрээс янз бүрийн хувиргалтуудаар дамжуулан эцсийн очих газар хүртэл мөшгих үйл явц юм. Энэ нь байгууллагын мэдээллийн архитектур дотор өгөгдлийн хөдөлгөөний иж бүрэн зураглалыг бий болгож, үндсэн асуултуудад хариулт өгдөг: *Энэ өгөгдөл хаанаас ирсэн бэ? Хэрхэн хувиргагдсан бэ? Хаана ашиглагддаг вэ?*

### 6.3.1 Өгөгдлийн гарал үүслийн төрлүүд

**Урагшлах гарал үүсэл (Нөлөөллийн шинжилгээ)** Урагшлах гарал үүсэл нь өгөгдлийг эх үүсвэрээс очих газар хүртэл мөшгиж, бүх доод түвшний хамаарлуудыг илрүүлдэг. Энэ нь өөрчлөлтийн нөлөөллийн шинжилгээнд маш чухал—эх хүснэгтийг өөрчлөхөд аль тайлан, хянах самбар эсвэл програмуудад нөлөөлөхийг ойлгох.

**Ухрах гарал үүсэл (Үндсэн шалтгааны шинжилгээ)** Ухрах гарал үүсэл нь өгөгдлийг одоогийн байршлаас эх үүсвэр хүртэл нь буцааж мөшгидөг. Энэ нь өгөгдлийн чанарын асуудал илэрсэн үед үндсэн шалтгааны шинжилгээг боломжтой болгож, инженерүүдэд асуудлууд хаанаас үүссэнийг яг таг тодорхойлох боломжийг олгодог.

**Хэвтээ гарал үүсэл** Хэвтээ гарал үүсэл нь ижил архитектурын түвшинд өгөгдлийн хөдөлгөөнийг харуулдаг, жишээлбэл өгөгдлийн агуулах доторх хүснэгтээс хүснэгт рүү хувиргалтууд.

### 6.3.2 Гарал үүслийн нарийвчлал

Өгөгдлийн гарал үүслийг олон нарийвчлалын түвшинд барьж авч болно:

- **Системийн түвшин:** Програмууд болон өгөгдлийн сангуудын хоорондын холболтууд.
- **Өгөгдлийн олонлогийн түвшин:** Хүснэгтүүд, файлууд, тайлангуудын хоорондын харилцаа.
- **Баганын түвшин:** Тус тусдаа талбарын хувиргалтуудын нарийвчилсан хяналт.

Орчин үеийн өгөгдлийн гарал үүслийн системүүд баганын түвшний гарал үүсэлд улам бүр анхаарч байгаа нь зохицуулалтын нийцэл болон дэлгэрэнгүй нөлөөллийн шинжилгээнд шаардлагатай нарийвчлалыг хангадаг. Бидний санал болгож буй систем нь гурван нарийвчлалын түвшинг бүгдийг дэмждэг бөгөөд хамгийн их ашиг тустай байхын тулд баганын түвшний хяналтад онцгой анхаарал хандуулдаг.

## 6.4 Хамрах хүрээ

Өгөгдлийн Гарал Үүслийн Системийн төсөл нь автоматжуулсан өгөгдлийн гарал үүслийн хяналтад зориулсан веб-д суурилсан програмын зохиомж, хөгжүүлэлт, байршуулалтыг хамарна. Хамрах хүрээ нь төслийн гаралт болон функцийн тодорхой хязгаарыг тодорхойлно.

### 6.4.1 Хамрах хүрээнд багтах

- **Метадата гаргаж авалт:** Дэмжигдсэн өгөгдлийн сангийн платформуудаас (PostgreSQL, MySQL, Oracle, SQL Server) SQL DDL/DML мэдэгдлүүд, хадгалагдсан процедурууд, view тодорхойлолтуудыг автоматаар задлан шинжлэх.
- **ETL интеграци:** Apache Airflow DAG-ууд болон тусгай Python скриптүүд зэрэг түгээмэл хэрэгслүүдээс ETL ажлын тодорхойлолтуудыг задлан шинжлэх.
- **Гарал үүслийн дүрслэл:** Томруулах, гүйлгэх, шүүх чадвартай өгөгдлийн урсгалын интерактив граф-д суурилсан дүрслэл.
- **Хайлт ба олох:** Талын шүүлтүүрийн сонголттой бүх метадата нэгжүүдээр бүрэн текст хайлт.
- **Хэрэглэгчийн удирдлага:** Үзэгч, засварлагч, администраторуудыг дэмждэг үүрэгт суурилсан хандалтын хяналт.
- **Экспортын функц:** Олон форматаар (PDF, PNG, JSON, CSV) гарал үүслийг экспортлох.
- **API хандалт:** Программчлалын гарал үүслийн асуулга болон интеграцид зориулсан RESTful API.

### 6.4.2 Хамрах хүрээнд багтахгүй

- Бодит цагийн урсгал гарал үүсэл барих (ирээдүйн хувилбарт төлөвлөгдсөн).
- Өгөгдлийн чанарын хяналт болон сэрэмжлүүлэг.
- Автоматжуулсан өгөгдлийн ангилал болон мэдрэмтгий байдлын шошголол.
- Арилжааны BI хэрэгслүүдтэй (Tableau, Power BI) интеграци.
- Гар утасны програм хөгжүүлэлт.

Энэ хамрах хүрээ нь ирээдүйн сайжруулалтын суурийг тавихын зэрэгцээ төвлөрсөн хөгжүүлэлтийг хангана.

## 6.5 Өгөгдлийн гарал үүслийн чухал тодорхойлолтууд

Өгөгдлийн гарал үүслийг ойлгоход гол нэр томъёотой танилцах шаардлагатай. Энэ хэсэгт энэ тайланд ашигласан чухал ойлголтуудыг тодорхойлно.

**Өгөгдлийн хөрөнгө (Data Asset)** Хүснэгт, view, файл, тайлан, ETL ажлууд зэрэг өгөгдөл агуулсан эсвэл боловсруулдаг аливаа нэгж.

**Эх үүсвэр (Source)** Гарал үүслийн харилцаан дахь өгөгдлийн эх цэг. Эх үүсвэр нь нэг буюу түүнээс олон зорилт руу урсдаг өгөгдлийг хангана.

**Зорилт (Target)** Гарал үүслийн харилцаан дахь өгөгдлийн очих газар. Зорилт нь нэг буюу түүнээс олон эх үүсвэрээс өгөгдөл хүлээн авна.

**Хувиргалт (Transformation)** Шүүлтүүр, нэгтгэл, холболт, тооцоолол зэрэг өгөгдлийг эх үүсвэрээс зорилт руу шилжих үед өөрчилдөг аливаа үйлдэл.

**Зангилаа (Node)** Өгөгдлийн хөрөнгийг төлөөлөх гарал үүслийн графын орой. Зангилаа бүр хөрөнгийн шинж чанарыг тодорхойлсон метадата агуулна.

**Ирмэг (Edge)** Хоёр зангилааны хооронд өгөгдлийн урсгалыг төлөөлөх гарал үүслийн графын холболт. Ирмэгүүд нь чиглэлтэй бөгөөд урсгалын чиглэлийг заана.

**Дээд урсгал (Upstream)** Өгөгдсөн хөрөнгөд өгөгдөл нийлүүлдэг бүх өгөгдлийн хөрөнгүүд. Дээд урсгалын шинжилгээ нь өгөгдлийн эх үүсвэрүүдийг илрүүлдэг.

**Доод урсгал (Downstream)** Өгөгдсөн хөрөнгөөс өгөгдөл хүлээн авдаг бүх өгөгдлийн хөрөнгүүд. Доод урсгалын шинжилгээ нь өгөгдлийн хэрэглэгчдийг илрүүлдэг.

**Метадата (Metadata)** Схемийн тодорхойлолт, тайлбар, эзэмшигч, техникийн шинж чанаруудыг багтаасан өгөгдлийн хөрөнгийн тухай тодорхойлолтын мэдээлэл.

**Схем (Schema)** Багануд, өгөгдлийн төрлүүд, хязгаарлалтууд, харилцааг тодорхойлсон өгөгдлийн сангийн объектын бүтцийн тодорхойлолт.

**ETL (Extract, Transform, Load)** Эх үүсвэрүүдээс өгөгдөл гаргаж авах, хувиргалтуудыг хэрэгжүүлэх, үр дүнг зорилт руу ачаалах өгөгдлийн интеграцийн үйл явц.

**DAG (Directed Acyclic Graph)** Чиглэлтэй ирмэгүүд бүхий, мөчлөггүй графын бүтэц бөгөөд өгөгдлийн дамжуулах хоолой болон гарал үүслийг төлөөлөхөд түгээмэл ашиглагддаг.

**Баганын түвшний гарал үүсэл (Column-Level Lineage)** Тус тусдаа эх баганыг хувиргалтуудаар дамжуулан зорилтын баганатай холбодог нарийвчилсан гарал үүслийн хяналт.

## 6.6 Зохиомж

Өгөгдлийн Гарал Үүслийн Систем нь өргөтгөх чадвар, засвар үйлчилгээний хялбар байдал, өргөтгөлийн боломжид зориулагдсан модульчлагдсан, микросервис-д суурилсан архитектурыг дагадаг. Энэ хэсэгт өндөр түвшний зохиомжийн зарчмууд болон архитектурын бүрэлдэхүүн хэсгүүдийг тоймлоно.

### 6.6.1 Архитектурын зарчмууд

- **Үүргийн салгалт:** Бүрэлдэхүүн хэсэг бүр тодорхой хариуцлагыг хариуцаж, бие даасан хөгжүүлэлт болон тестлэлтийг боломжтой болгоно.
- **Өргөтгөх чадвар:** Плагин-д суурилсан задлагчийн архитектур нь үндсэн системийг өөрчлөхгүйгээр шинэ өгөгдлийн эх үүсвэрүүдийн дэмжлэг нэмэх боломжийг олгоно.
- **Масштаблах чадвар:** Төлөвгүй API үйлчилгээнүүд нь нэмэгдсэн метадата хэмжээг зохицуулахын тулд хэвтээ масштаблалтыг дэмждэг.

– **Аюулгүй байдлын зохиомж:** Баталгаажуулалт, зөвшөөрөл, аудит бүртгэл нь архитектурын түвшинд интеграцилагдсан.

### 6.6.2 Системийн бүрэлдэхүүн хэсгүүд

**Танилцуулгын давхарга** Хэрэглэгчийн интерфэйсийг хангадаг React-д суурилсан нэг хуудсан програм (SPA). Функцууд нь D3.js ашигласан интерактив гарал үүслийн графууд, метадата үзэх, удирдлагын функцуудыг багтаана.

**API давхарга** Бизнесийн логик, баталгаажуулалт, өгөгдлийн хандалтыг зохицуулдаг Node.js/Express-ээр бүтээгдсэн RESTful API үйлчилгээнүүд. GraphQL endpoint-ууд нь гарал үүслийн дамжуулалтад уян хатан асуулга хийх боломжийг олгоно.

**Задлагчийн хөдөлгүүр** SQL задлан шинжлэлд ANTLR, ETL хэрэгслүүдэд тусгай задлагчид ашигладаг Python-д суурилсан задлан шинжлэлийн фреймворк. Задлагчид нь эх үүсвэрийн бүтээгдэхүүнүүдээс гарал үүслийн харилцаа болон метадатаг гаргаж авна.

**Граф өгөгдлийн сан** Neo4j нь гарал үүслийн харилцааны үндсэн хадгалалт болж, үр ашигтай дамжуулалтын асуулгад уугуул графын чадамжуудыг ашигладаг. Харилцааны өгөгдлийн санд үнэтэй join шаарддаг харилцааны асуулгууд миллисекундэд биелдэг.

**Метадата хадгалах сан** PostgreSQL нь нэмэлт метадата, хэрэглэгчийн мэдээлэл, аудит бүртгэл, харилцааны бүтцээс ашиг хүртдэг тохиргооны өгөгдлийг хадгална.

### 6.6.3 Технологийн стек

Table 17: Технологийн стекийн тойм

| **Давхарга** | **Технологи** |
|---|---|
| Frontend | React, TypeScript, D3.js |
| Backend | Node.js, Express, Python |
| Граф өгөгдлийн сан | Neo4j |
| Харилцааны өгөгдлийн сан | PostgreSQL |
| Баталгаажуулалт | JWT, OAuth 2.0 |
| Контейнержуулалт | Docker, Kubernetes |

## 6.7 Төслийн суурь мэдээлэл

Өгөгдлийн Гарал Үүслийн Системийн төсөл нь байгууллагын өгөгдлийн удирдлагын чадамжийн чухал цоорхойгоос үүссэн. Байгууллагууд өгөгдөлд суурилсан шийдвэр гаргалтад улам бүр найддаг болохын хэрээр тэдний өгөгдлийн экосистемийн нарийн төвөгтэй байдал экспоненциалаар өссөн. Энэ хэсэгт төслийг салбарын өргөн хандлага болон байгууллагын хэрэгцээний хүрээнд тайлбарлана.

### 6.7.1 Салбарын нөхцөл байдал

Дэлхийн өгөгдлийн засаглалын зах зээл нь зохицуулалтын дарамт (GDPR, CCPA, SOX) болон өгөгдлийн хөрөнгийн стратегийн ач холбогдлоос үүдэн 2025 он гэхэд 5.7 тэрбум ам.долларт хүрэхээр төлөвлөгдөж байна. Энэ зах зээлийн дотор өгөгдлийн гарал үүслийн хэрэгслүүд нь хамгийн хурдан өсөж буй сегментүүдийн нэгийг төлөөлж, байгууллагууд гарын авлагын баримтжуулалтын практикийг орлох автоматжуулсан шийдлүүдийг хайж байна.

Гарал үүслийн баримтжуулалтын уламжлалт аргууд—хүснэгтүүд, wiki хуудсууд, хүмүүсийн мэдлэг—орчин үеийн өгөгдлийн орчинд хангалтгүй болохыг нотолсон. Гарын авлагын баримтжуулалт хурдан хуучирдаг, нийцлийн шаардлагатай нарийвчлал дутагддаг, өгөгдлийн хэмжээ нэмэгдэхэд муу масштаблагддаг.

### 6.7.2 Байгууллагын сорилтууд

Энэ төслийг санаачлахад хүргэсэн ердийн байгууллагын сорилтууд:

- **Тусгаарлагдсан метадата:** Гарал үүслийн мэдээлэл олон хэрэгсэл болон багуудад тархсан, нэгдсэн харагдац байхгүй.
- **Гарын авлагын баримтжуулалтын ачаалал:** Өгөгдлийн инженерүүд цагийнхаа 20-30%-ийг хөгжүүлэлтийн оронд баримтжуулалтад зарцуулдаг.
- **Нийцлийн эрсдэл:** Зохицуулалтын аудитын үеэр өгөгдлийн гарал үүслийг харуулж чадахгүй байдал.
- **Өөрчлөлтийн удирдлагын алдаа:** Доод түвшний нөлөөллийг ойлгохгүйгээр өөрчлөлт хийснээс үүссэн үйлдвэрлэлийн ослууд.
- **Өгөгдөл олох үр ашиггүй байдал:** Аналистууд холбогдох өгөгдлийн олонлогуудыг олж чадахгүй байгаа нь давхар ажилд хүргэдэг.

### 6.7.3 Төслийн үүсэл

Энэ төсөл нь автоматжуулалт болон дүрслэлээр дамжуулан эдгээр сорилтуудыг шийдвэрлэхэд санаачлагдсан. Шийдэл нь гарын авлагын гарал үүслийн баримтжуулалтыг устгах, өгөгдлийн урсгалд бодит цагийн харагдах байдлыг хангах, байгууллага даяарх метадатагийн эрх мэдэл бүхий эх үүсвэр болохыг зорьж байна.

## 6.8 Санал болгож буй шийдэл

Санал болгож буй Өгөгдлийн Гарал Үүслийн Систем нь гарал үүслийн барилтыг автоматжуулж, зөн совинтой дүрслэл хангаж, одоо байгаа өгөгдлийн дэд бүтэцтэй интеграцилагдсан иж бүрэн веб-д суурилсан платформ юм. Энэ хэсэгт шийдлийн гол чадамжууд болон ялгарах онцлогуудыг тайлбарлана.

### 6.8.1 Шийдлийн тойм

Систем нь метадата цуглуулалт, задлан шинжлэл, хадгалалт, танилцуулгын тасралтгүй мөчлөгөөр ажилладаг:

1. **Цуглуулалт:** Автоматжуулсан агентууд тохируулагдсан өгөгдлийн эх үүсвэрүүдийг сканнерддаг, DDL скриптүүд, асуулгын бүртгэлүүд, ETL тодорхойлолтуудыг гаргаж авна.
2. **Задлан шинжлэл:** Мэргэжлийн задлагчид цуглуулсан бүтээгдэхүүнүүдийг шинжилж, нэгжүүд, харилцаанууд, хувиргалтуудыг тодорхойлно.
3. **Хадгалалт:** Задлан шинжилсэн гарал үүслийг харилцааны дамжуулалтад оновчлогдсон граф өгөгдлийн санд хадгална.
4. **Танилцуулга:** Хэрэглэгчид интерактив графууд болон хайлтыг онцолсон зөн совинтой веб интерфэйсээр гарал үүсэлтэй харилцана.

### 6.8.2 Гол онцлогууд

**Автоматжуулсан гарал үүслийн гаргаж авалт** Систем нь гарын авлагын оролт шаардахгүйгээр хүснэгт болон баганын түвшний гарал үүслийг гаргаж авахын тулд SQL мэдэгдлүүдийг автоматаар задлан шинжилдэг. CTE-үүд, дэд асуулгууд, цонхны функцууд зэрэг нарийн төвөгтэй асуулгуудын дэмжлэг нь иж бүрэн хамрах хүрээг хангана.

**Интерактив дүрслэл** Хүчний чиглэлтэй графын дүрслэл нь хэрэглэгчдэд гарал үүслийн харилцааг зөн совиноор судлах боломжийг олгоно. Функцууд нь дараах зүйлсийг багтаана:

- Томруулах болон гүйлгэх навигаци
- Зангилааны бүлгүүдийг өргөтгөх/хураах
- Дээд/доод урсгалын шүүлтүүр
- Зангилааны хайлт болон тодруулалт

**Нөлөөллийн шинжилгээ** Өөрчлөлт хийхээс өмнө хэрэглэгчид өөрчлөлтүүдийг симуляцилж, нөлөөлөх бүх доод түвшний хөрөнгүүдийг дүрслэн харах боломжтой. Энэ нь өөрчлөлтүүдийг хэрэгжүүлэхээс өмнө бүрэн ойлгохыг баталгаажуулснаар үйлдвэрлэлийн ослуудыг бууруулдаг.

**API-д түрүүлсэн зохиомж** Бүх функц RESTful API-уудаар хандагдах боломжтой бөгөөд CI/CD дамжуулах хоолой, өгөгдлийн каталогууд, тусгай програмуудтай интеграцийг боломжтой болгоно.

### 6.8.3 Байршуулалтын загвар

Шийдэл нь янз бүрийн дэд бүтцийн шаардлагатай байгууллагуудад уян хатан байдлыг хангаж, байгууллага дээрх болон үүлэн байршуулалтыг (AWS, Azure, GCP) хоёуланг нь дэмждэг.

## 6.9 Шаардлагын жагсаалт

Энэ хэсэгт Өгөгдлийн Гарал Үүслийн Системийн функциональ болон функциональ бус шаардлагууд, мөн таамаглалууд, хамаарлууд, тодорхойлогдсон эрсдэлүүдийг баримтжуулна.

### 6.9.1 Шаардлагын тодорхойлолт

Table 18: Функциональ шаардлагууд

| ID | Шаардлага |
|---|---|
| FR-01 | Систем нь гарал үүслийн харилцааг гаргаж авахын тулд SQL DDL болон DML мэдэгдлүүдийг задлан шинжлэх ёстой. |
| FR-02 | Систем нь баганын түвшний гарал үүслийн хяналтыг дэмжих ёстой. |
| FR-03 | Систем нь гарал үүслийн интерактив графын дүрслэл хангах ёстой. |
| FR-04 | Систем нь дээд болон доод урсгалын гарал үүслийн дамжуулалтыг дэмжих ёстой. |
| FR-05 | Систем нь үүрэгт суурилсан хандалтын хяналтыг (RBAC) хэрэгжүүлэх ёстой. |
| FR-06 | Систем нь PDF, PNG, JSON, CSV форматуудаар гарал үүслийг экспортлох ёстой. |
| FR-07 | Систем нь бүх метадатаар бүрэн текст хайлт хангах ёстой. |
| FR-08 | Систем нь бүх хэрэглэгчийн үйлдлийн аудит бүртгэл хөтлөх ёстой. |
| FR-09 | Систем нь автоматжуулсан метадата сканнерын хуваарь тохируулахыг дэмжих ёстой. |
| FR-10 | Систем нь гадаад интеграцид зориулсан RESTful API-уудыг хангах ёстой. |

#### Функциональ шаардлагууд

#### Функциональ бус шаардлагууд

- **Гүйцэтгэл:** Гарал үүслийн асуулгууд 10,000 хүртэлх зангилаа бүхий графуудад 2 секундын дотор үр дүнг буцаах ёстой.
- **Масштаблах чадвар:** Систем нь доройтолгүйгээр 1 сая хүртэлх өгөгдлийн хөрөнгийг дэмжих ёстой.
- **Бэлэн байдал:** Ажлын цагаар 99.5% uptime зорилт.
- **Аюулгүй байдал:** Дамжуулалт дахь бүх өгөгдлийг TLS 1.2+ ашиглан шифрлэх ёстой.
- **Хэрэглэхэд хялбар байдал:** Шинэ хэрэглэгчид сургалтгүйгээр 5 минутын дотор гарал үүслийг удирдах чадвартай байх ёстой.

### 6.9.2 Таамаглал ба хамаарлууд

**Таамаглалууд**

- Эх өгөгдлийн сангууд системийн каталог болон мэдээллийн схемд унших хандалт хангадаг.
- SQL скриптүүд стандарт ANSI SQL синтакс эсвэл дэмжигдсэн диалектын хувилбаруудыг дагадаг.
- Хэрэглэгчид JavaScript идэвхжсэн орчин үеийн веб хөтчүүдтэй (Chrome, Firefox, Safari, Edge).
- Гарал үүслийн систем болон бүх хянагдаж буй өгөгдлийн эх үүсвэрүүдийн хооронд сүлжээний холболт байдаг.

**Хамаарлууд**

- Neo4j Community эсвэл Enterprise Edition (v4.4+)
- PostgreSQL (v13+)
- Node.js Runtime (v16+)
- Python (v3.9+) ANTLR runtime-тай
- Контейнержуулсан байршуулалтад Docker ба Kubernetes

### 6.9.3 Эрсдэл

Table 19: Эрсдэлийн үнэлгээ

| Эрсдэл | Магадлал | Нөлөө | Бууруулах арга |
|---|---|---|---|
| Задлагч нарийн төвөгтэй SQL-д алдагдах | Дунд | Өндөр | Нөөц гарын авлагын гарал үүслийн оруулалт хэрэгжүүлэх; дүрмийн хамрах хүрээг өргөтгөх |
| Масштабд гүйцэтгэл буурах | Бага | Өндөр | Асуулгын оновчлол хэрэгжүүлэх; кэш давхарга нэмэх |
| Аюулгүй байдлын сул тал | Дунд | Өндөр | Тогтмол аюулгүй байдлын аудит; нэвтрэлтийн тест |
| Хэрэглэгчийн нэвтрүүлэлтийн эсэргүүцэл | Дунд | Дунд | UX дизайнд хөрөнгө оруулах; сургалт явуулах |
| Интеграцийн алдаа | Бага | Дунд | Иж бүрэн API тест; баримтжуулалт |

## 6.10 UML диаграммууд

Энэ хэсэгт Өгөгдлийн Гарал Үүслийн Системийн бүтэц болон зан төлөвийг тодорхойлсон гол UML диаграммуудыг танилцуулна. Диаграмм бүрийг текстээр тайлбарлаж, хялбаршуулсан дүрслэлтэй хамт өгнө.

### 6.10.1 Use Case диаграмм

Use Case диаграмм нь үндсэн оролцогчид болон тэдний системтэй харилцааг тодорхойлно. Гурван үндсэн оролцогч Өгөгдлийн Гарал Үүслийн Системтэй харилцдаг: **Өгөгдлийн инженер** (эх үүсвэрүүдийг тохируулах, сканнер эхлүүлэх), **Өгөгдлийн аналист** (гарал үүслийг үзэх, хайлт хийх), **Администратор** (хэрэглэгчдийг удирдах, системийн тохиргоо).

Гол use case-үүд нь: Гарал үүсэл үзэх, Хөрөнгө хайх, Нөлөөллийн шинжилгээ харах, Өгөгдлийн эх үүсвэр тохируулах, Сканнер эхлүүлэх, Гарал үүсэл экспортлох, Хэрэглэгчдийг удирдах, Аудит бүртгэл харах. Өгөгдлийн инженер болон Аналист нь гарал үүсэл үзэх нийтлэг use case-үүдийг хуваалцдаг бол удирдлагын функцууд нь Администраторын үүрэгт хязгаарлагддаг.

```
+-----------------------------------------------------+
|     Өгөгдлийн Гарал Үүслийн Систем                   |
|                                                      |
|   (Гарал үүсэл үзэх)----+                           |
|   (Хөрөнгө хайх)--------+----> [Өг. Аналист]         |
|   (Нөлөөлөл харах)------+                           |
|                                                      |
|   (Эх үүсвэр тохируулах)+                           |
|   (Сканнер эхлүүлэх)----+----> [Өг. Инженер]         |
|   (Гарал үүсэл экспорт)-+                           |
|                                                      |
|   (Хэрэглэгч удирдах)---+                           |
|   (Аудит бүртгэл харах)-+----> [Администратор]       |
|   (Системийн тохиргоо)--+                           |
+-----------------------------------------------------+
```

### 6.10.2 Class диаграмм

Class диаграмм нь системийн үндсэн домэйн моделийг харуулна. Гол ангиуд нь `DataAsset` (бүх өгөгдлийн нэгжийн хийсвэр суурь), `Table`, `Column`, `ETLJob`, `Report` юм. `LineageEdge` анги нь хөрөнгүүдийн хоорондын харилцааг төлөөлж, хувиргалтын төрөл болон итгэлцлийн оноо зэрэг шинж чанаруудтай.

`User` анги нь хандалтын хяналтад `Role`-той холбогддог. `DataSource` нь тохируулагдсан гадаад системүүдийг төлөөлдөг бол `ScanJob` нь метадата гаргаж авалтын ажиллагааг хянадаг.

```
+----------------+       +------------------+
|   DataAsset    |<------|   LineageEdge    |
+----------------+       +------------------+
| -id: String    |       | -sourceId: String|
| -name: String  |       | -targetId: String|
| -type: String  |       | -transform: String|
| -metadata: Map |       +------------------+
+----------------+
```

```
       ^
       |
+------+-------+--------+
|              |        |
+--------+ +--------+ +--------+
| Table  | | ETLJob | | Report |
+--------+ +--------+ +--------+

+--------+     +--------+
|  User  |---->|  Role  |
+--------+     +--------+
```

### 6.10.3 Activity диаграмм

Activity диаграмм нь гарал үүслийн гаргаж авалтын ажлын урсгалыг дүрсэлнэ. Үйл явц нь хэрэглэгч эсвэл хуваарилагч сканнер эхлүүлэх үед эхэлнэ. Систем нь тохируулагдсан өгөгдлийн эх үүсвэрт холбогдож, DDL/DML мэдэгдлүүдийг гаргаж авч, задлагчийн хөдөлгүүрт дамжуулна.

Задлагч нь оролтыг токенжуулж, AST (Abstract Syntax Tree) байгуулж, эх үүсвэр болон зорилтын нэгжүүдийг тодорхойлж, баганын зураглалыг гаргаж авна. Задлан шинжилсэн харилцаануудыг одоо байгаа метадататай харьцуулан баталгаажуулж, дараа нь граф өгөгдлийн санд хадгална. Эцэст нь шинэ гарал үүслийг тусгахын тулд UI кэшийг хүчингүй болгоно.

```
[Эхлэл]
   |
   v
(Сканнер эхлүүлэх)
   |
   v
(Өгөгдлийн эх үүсвэрт холбогдох)
   |
   v
(SQL/DDL гаргаж авах)
   |
   v
(Мэдэгдлүүдийг задлах)-----> <Шийдвэр: Задлалт амжилттай?>
   |                                |
   | Тийм                           | Үгүй
   v                                v
(Харилцаануудыг гаргах)       (Алдаа бүртгэх)
   |                                |
   v                                |
(Гарал үүслийг баталгаажуулах)      |
   |                                |
   v                                |
(Граф ӨС-д хадгалах)               |
   |                                |
```

```
   v                                |
(Кэш хүчингүй болгох) <-----------+
   |
   v
[Төгсгөл]
```

### 6.10.4 Sequence диаграмм

Sequence диаграмм нь хэрэглэгч тодорхой хүснэгтийн гарал үүслийг хүсэх үеийн харилцааны урсгалыг харуулна. Хэрэглэгч UI-ээр хүсэлтийг эхлүүлж, UI нь API Gateway-г дууддаг. Gateway нь Auth Service-ээр хүсэлтийг баталгаажуулж, дараа нь асуулгыг Lineage Service руу дамжуулна.

Lineage Service нь Cypher асуулгууд ашиглан Neo4j-ээс дээд болон доод урсгалын харилцаануудыг асууна. Үр дүнг зангилаа болон ирмэгүүдийг агуулсан хариултын объект болгон угсарна. Хариулт нь Gateway-ээр UI руу буцаж очиж, интерактив графыг рендэрлэнэ.

```
Хэрэглэгч    UI        Gateway    AuthService   LineageService   Neo4j
 |           |            |            |              |            |
 |--Хүсэлт-->|            |            |              |            |
 |           |--API Call->|            |              |            |
 |           |            |--Баталгаа->|              |            |
 |           |            |<--Token OK-|              |            |
 |           |            |--Гарал үүсэл авах-------->|            |
 |           |            |            |              |--Асуулга-->|
 |           |            |            |              |<--Зангилаа-|
 |           |            |<--Хариулт-----------------|            |
 |           |<--JSON-----|            |              |            |
 |<--Рендэр--|            |            |              |            |
 |           |            |            |              |            |
```

## 6.11 Хэрэгжүүлэлт

Энэ хэсэгт Өгөгдлийн Гарал Үүслийн Системийн гурван үндсэн модулийн хэрэгжүүлэлтийг дэлгэрэнгүй тайлбарлана: Хэрэглэгчийн модуль, Задлагч, Графикийн хэрэглэгчийн интерфэйс.

### 6.11.1 Хэрэглэгчийн модуль

Хэрэглэгчийн модуль нь Өгөгдлийн Гарал Үүслийн Системд иж бүрэн хэн мөн болохыг удирдах, баталгаажуулалт, зөвшөөрлийн чадамжуудыг хангадаг. Энэ нь байгууллагын шаталсан бүтэц болон үүрэгт суурилсан зөвшөөрлүүдийг дэмжихийн зэрэгцээ аюулгүй хандалтыг хангадаг.

**Зорилго ба онцлогууд** Модуль нь бүртгэлээс идэвхгүй болгох хүртэлх хэрэглэгчийн амьдралын мөчлөгийг удирдаж, профайл, итгэмжлэл, хандалтын зөвшөөрлүүдийг хадгалдаг. Гол функцууд нь:

- Имэйл баталгаажуулалттай хэрэглэгчийн бүртгэл
- bcrypt хэшлэлт ашигласан аюулгүй нууц үг хадгалалт
- JWT токенуудаар сесс удирдлага
- Нууц үг сэргээх функц
- Профайл удирдлага (нэр, имэйл, тохиргоо)
- Аудит нийцлийн үйл ажиллагааны бүртгэл

**Нэвтрэлт ба баталгаажуулалт** Баталгаажуулалт нь байгууллагын нийцтэй байдлын OAuth 2.0 / OpenID Connect стандартуудыг дагадаг. Систем нь олон баталгаажуулалтын аргуудыг дэмждэг:

- **Дотоод баталгаажуулалт:** Хэрэглэгчийн нэр/нууц үг, нэмэлт MFA (TOTP)-тай
- **SSO интеграци:** Байгууллагын хэн мөн болохын үйлчилгээ үзүүлэгчдэд зориулсан SAML 2.0
- **API баталгаажуулалт:** Программчлалын хандалтад зориулсан API түлхүүрүүд болон JWT bearer токенууд

JWT токенууд нь хэрэглэгчийн хэн мөн болох, үүрэг, дуусах хугацааны мэдээллийг агуулдаг. Хандалтын токенууд 1 цагийн дараа дуусдаг; шинэчлэх токенууд нь 7 хүртэлх хоногт саадгүй дахин баталгаажуулалтыг боломжтой болгодог.

**Хэрэглэгчийн үүргүүд** Систем нь шаталсан зөвшөөрөл бүхий гурван урьдчилан тодорхойлсон үүргийг хэрэгжүүлдэг:

- **Үзэгч:** Гарал үүслийн графууд болон метадатад зөвхөн унших хандалт. Тохиргоог өөрчлөх эсвэл өгөгдөл экспортлох боломжгүй.
- **Засварлагч:** Үзэгчийн бүрэн зөвшөөрөл дээр гарын авлагын гарал үүслийн оруулга нэмэх, хөрөнгө тэмдэглэх, өгөгдөл экспортлох чадвартай.
- **Администратор:** Хэрэглэгчийн удирдлага, өгөгдлийн эх үүсвэрийн тохиргоо, системийн тохиргоо зэрэг системд бүрэн хандалт.

**Гол функцууд** Үндсэн API endpoint-ууд:

- `POST /auth/login` — Хэрэглэгчийн итгэмжлэлийг баталгаажуулах
- `POST /auth/refresh` — Хандалтын токеныг шинэчлэх
- `GET /users/me` — Одоогийн хэрэглэгчийн профайлыг авах
- `PUT /users/:id` — Хэрэглэгчийн мэдээллийг шинэчлэх
- `GET /users/:id/activity` — Хэрэглэгчийн үйл ажиллагааны бүртгэлийг авах

### 6.11.2 Задлагч

Задлагч модуль нь Өгөгдлийн Гарал Үүслийн Системийн аналитик хөдөлгүүр бөгөөд олон төрлийн өгөгдлийн эх үүсвэрүүдээс метадата болон гарал үүслийн харилцааг гаргаж авах үүрэгтэй. Энэ нь түүхий SQL, ETL скриптүүд, схемийн тодорхойлолтуудыг бүтэцлэгдсэн гарал үүслийн графууд болгон хувиргадаг.

**Метадата гаргаж авах үйл явц** Задлагч нь олон үе шаттай гаргаж авах дамжуулах хоолой ашигладаг:

1. **Эх үүсвэрийн холболт:** Тохируулагдсан итгэмжлэл ашиглан зорилтот өгөгдлийн сан эсвэл файлын системд аюулгүй холболт үүсгэх.
2. **Бүтээгдэхүүн цуглуулалт:** Системийн каталогуудаас (`information_schema`), асуулгын бүртгэлүүд, файлын хадгалах сангуудаас DDL мэдэгдлүүдийг татаж авах.
3. **Лексик шинжилгээ:** SQL диалект бүрт зориулсан ANTLR-ээр үүсгэгдсэн лексерүүд ашиглан оролтыг токенжуулах.
4. **Синтакс задлан шинжлэл:** Асуулгын бүтцийг төлөөлсөн Abstract Syntax Trees (AST) байгуулах.
5. **Семантик шинжилгээ:** Хүснэгт/баганын лавлагаануудыг шийдвэрлэх, alias-уудыг зохицуулах, хувиргалтуудыг тодорхойлох.
6. **Гарал үүсэл гаргаж авалт:** Хувиргалтын логикоор дамжуулан эх баганыг зорилтын баганатай зураглах.

**Дэмжигдсэн форматууд** Задлагч нь дараах оролтын форматуудыг дэмждэг:

- **SQL диалектууд:** PostgreSQL, MySQL, Oracle PL/SQL, T-SQL (SQL Server), ANSI SQL
- **DDL мэдэгдлүүд:** CREATE TABLE, CREATE VIEW, ALTER TABLE
- **DML мэдэгдлүүд:** SELECT, INSERT, UPDATE, MERGE, CTAS
- **ETL тодорхойлолтууд:** Apache Airflow DAG-ууд (Python), dbt моделүүд (SQL + YAML)
- **Схем файлууд:** JSON Schema, Avro, Parquet метадата

**Гаралтын формат** Задлан шинжилсэн гарал үүсэл нь стандартчилагдсан схемд нийцсэн JSON баримт бичиг хэлбэрээр гарна:

```
{
  "nodes": [
    {
      "id": "source_db.schema.table",
      "type": "TABLE",
      "columns": ["col1", "col2"]
    }
  ],
  "edges": [
    {
```

```
      "source": "source_db.schema.table.col1",
      "target": "target_db.schema.table.col_a",
      "transformation": "DIRECT_COPY",
      "confidence": 1.0
    }
  ]
}
```

Listing 4: Гарал үүслийн гаралтын схем

### 6.11.3 Графикийн хэрэглэгчийн интерфэйс

Графикийн хэрэглэгчийн интерфэйс (GUI) нь гарал үүслийн өгөгдөлтэй харилцах зөн совинтой, веб-д суурилсан frontend-ийг хангадаг. Энэ нь гарын товчлол болон дэвшилтэт шүүлтүүрээр дамжуулан хүчирхэг хэрэглэгчийн ажлын урсгалыг дэмжихийн зэрэгцээ харааны судлалыг онцолдог.

**Фреймворк ба технологи** GUI нь орчин үеийн веб технологиуд ашиглан бүтээгдсэн:

- **React 18:** Төлөв удирдлагад hooks бүхий бүрэлдэхүүн хэсэгт суурилсан UI фреймворк
- **TypeScript:** Сайжруулсан засвар үйлчилгээнд статик тайпчлал
- **D3.js:** Гарал үүслийн графуудад зориулсан өгөгдөлд суурилсан дүрслэлийн сан
- **Material-UI:** Тогтвортой дизайны хэлийг хангадаг бүрэлдэхүүн хэсгийн сан
- **React Query:** Кэшлэлттэй серверийн төлөв удирдлага

**Үндсэн дэлгэцүүд** Интерфэйс нь хэд хэдэн гол дэлгэцээс бүрдэнэ:

- **Хянах самбар:** Сүүлийн сканнерууд, гарал үүслийн статистик, хурдан хайлтын тойм.
- **Каталог хөтөч:** Эх үүсвэр, схем, төрлөөр өгөгдлийн хөрөнгүүдийн шаталсан навигаци.
- **Гарал үүслийн судлагч:** Интерактив графтай үндсэн дүрслэлийн дэлгэц.
- **Хайлтын үр дүн:** Хөрөнгийн төрөл, эх үүсвэр, шошгоор талын шүүлтүүр бүхий бүрэн текст хайлт.
- **Хөрөнгийн дэлгэрэнгүй:** Бие даасан хөрөнгийн метадата, гарал үүсэл, хэрэглээний статистикийн иж бүрэн харагдац.
- **Удирдлага:** Хэрэглэгчийн удирдлага, эх үүсвэрийн тохиргоо, системийн тохиргоо.

**Гарал үүслийн графын дүрслэл** GUI-ийн голд интерактив гарал үүслийн граф байна. Функцууд нь:

- Органик зангилааны байршуулалтад физикийн симуляци бүхий хүчний чиглэлтэй зохион байгуулалт
- Томруулах (хулганы дугуй) болон гүйлгэх (чирэх) навигаци
- Холбогдсон зангилаануудыг аажмаар илрүүлэхийн тулд дарж өргөтгөх
- Хөрөнгийн төрлөөр өнгөний код (хүснэгтүүд = цэнхэр, view-үүд = ногоон, ETL = улбар шар)
- Хувиргалтын төрлүүдийг харуулсан ирмэгийн шошго
- Том графуудад чиглүүлэхэд зориулсан жижиг зураг

**Интерактив функцууд** Хэрэглэгчид графтай дараах байдлаар харилцаж болно:

- **Төвлөрлийн горим:** Сонгосон зангилаанаас дээд эсвэл доод урсгалын замуудыг тодруулах
- **Шүүлтүүрийн самбар:** Төрөл, эх үүсвэр, хайлтын шалгуураар зангилаануудыг харуулах/нуух
- **Дэлгэрэнгүй хажуугийн самбар:** Графаас гарахгүйгээр сонгосон зангилааны метадатаг харах
- **Тэмдэглэгээ:** Хамтын ажиллагаанд зориулж зангилаанд тайлбар болон шошго нэмэх

**Экспортын сонголтууд** Гарал үүслийг баримтжуулалт болон хуваалцахад экспортлох боломжтой:

- **PNG/SVG:** Одоогийн графын харагдацын өндөр нарийвчлалтай зураг экспорт
- **PDF:** Гарал үүслийн граф болон метадатаг багтаасан форматлагдсан тайлан
- **JSON:** Программчлалын хэрэглээнд зориулсан машин уншигдах гарал үүсэл
- **CSV:** Зангилаа болон ирмэгийн жагсаалтын хүснэгт экспорт

## 6.12 Дүгнэлт

Энэ тайланд танилцуулсан Өгөгдлийн Гарал Үүслийн Систем нь орчин үеийн өгөгдлийн удирдлагын чухал хэрэгцээг хангадаг: нарийн төвөгтэй байгууллагын орчинд өгөгдлийн урсгалыг автоматжуулсан хяналт. Ухаалаг задлан шинжлэл, граф-д суурилсан хадгалалт, зөн совинтой дүрслэлийг хослуулснаар систем нь тодорхойгүй өгөгдлийн экосистемүүдийг ил тод, навигацилах боломжтой орчин болгон хувиргадаг.

### 6.12.1 Амжилтуудын товч

Энэ төсөл амжилттайгаар дараах зүйлсийг хүргүүлсэн:

- Олон өгөгдлийн эх үүсвэрүүдээс автоматжуулсан гарал үүслийн гаргаж авалтыг дэмждэг иж бүрэн архитектур.
- Гол өгөгдлийн сангийн платформуудаар баганын түвшний нарийвчлалтай SQL-ийг шинжлэх чадвартай боловсронгуй задлагчийн хөдөлгүүр.
- Гарал үүслийг техникийн болон техникийн бус хэрэглэгчдэд хоёуланд нь хүртээмжтэй болгодог интерактив графын дүрслэл бүхий зөн совинтой веб интерфэйс.
- Байгууллагын байршуулалтад тохирсон аюулгүй, үүрэгт суурилсан хандалтын хяналтын систем.
- Одоо байгаа өгөгдлийн дэд бүтэц болон CI/CD дамжуулах хоолойтой интеграцийг боломжтой болгодог RESTful API-ууд.

### 6.12.2 Бизнесийн үнэ цэнэ

Энэ системийг хэрэгжүүлж буй байгууллагууд дараах зүйлсийг хүлээж болно:

- Гарын авлагын гарал үүслийн баримтжуулалтад зарцуулах цагийг 60-70%-иар бууруулах
- Аудит хийгдэх боломжтой гарал үүслийн бүртгэлээр дамжуулан зохицуулалтын нийцлийг сайжруулах
- Идэвхтэй нөлөөллийн шинжилгээгээр дамжуулан үйлдвэрлэлийн ослуудыг бууруулах
- Аналитик санаачилгуудыг хурдасгах сайжруулсан өгөгдөл олох чадвар

### 6.12.3 Ирээдүйн сайжруулалтууд

Дараах сайжруулалтуудыг ирээдүйн хувилбаруудад төлөвлөж байна:

1. **Бодит цагийн гарал үүсэл:** Kafka, Spark Streaming болон ижил төстэй платформуудаас гарал үүсэл барихад зориулсан урсгал боловсруулалтын интеграци.
2. **Машин сургалтын сайжруулалт:** Баримтжуулалт муутай хуучин системүүдэд зориулсан AI-д туслагдсан гарал үүслийн дүгнэлт.
3. **Өгөгдлийн чанарын интеграци:** Асуудал автоматаар тархах сэрэмжлүүлэг бүхий гарал үүсэлд мэдрэмтгий өгөгдлийн чанарын хяналт.
4. **BI хэрэгслийн холбогчид:** Тайлангийн түвшний гарал үүсэлд зориулсан Tableau, Power BI, Looker-тэй уугуул интеграци.
5. **Хамтын ажиллагааны функцууд:** Гарал үүслийн өөрчлөлтөд зориулсан тайлбар, мэдэгдэл, ажлын урсгалын батлал.
6. **Гар утасны програм:** Явж байхдаа гарал үүсэлд хандахад зориулсан iOS болон Android програмууд.

Өгөгдлийн Гарал Үүслийн Систем нь иж бүрэн өгөгдлийн засаглалын суурийг бий болгож, байгууллагуудыг өөрсдийн өгөгдлийн хөрөнгийг итгэлтэй, тодорхой байдлаар удирдах боломжийг олгоно.

## 6.13 Ном зүй


1. Thompson, R., & Garcia, M. (2019). Байгууллагын өгөгдлийн засаглалд зориулсан автоматжуулсан метадата удирдлагын фреймворкууд. *Journal of Data Management*, 15(3), 234–251.
2. Chen, L., Wang, H., & Zhang, Y. (2020). Нарийн төвөгтэй өгөгдлийн гарал үүслийн системүүдэд зориулсан граф-д суурилсан дүрслэлийн техникүүд. *IEEE Transactions on Visualization and Computer Graphics*, 26(8), 2891–2904.
3. Williams, J., & Brown, S. (2018). Байгууллагын өгөгдлийн гарал үүслийн интеграцийн сорилтууд: Олон платформын судалгаа. *International Journal of Information Management*, 42, 78–92.
4. Kumar, A., & Patel, R. (2021). Автомат SQL гарал үүсэл гаргаж авахад машин сургалтын аргууд. *Proceedings of the ACM SIGMOD Conference*, 1156–1168.
5. Anderson, K. (2020). GDPR болон CCPA-ийн эрин дэх өгөгдлийн гарал үүслийн зохицуулалтын үр нөлөө. *Journal of Data Protection & Privacy*, 4(2), 112–128.
6. Liu, X., & Yamamoto, T. (2022). Урсгал өгөгдлийн архитектурт үйл явдалд суурилсан гарал үүсэл барих. *Proceedings of the IEEE International Conference on Big Data*, 892–901.
7. Martinez, C. (2019). *Өгөгдлийн засаглал: Орчин үеийн байгууллагын зарчим ба практик.* O'Reilly Media.

# A Программын кодын хэсгээс

## A.1 Захиалга хүлээн авах функц (Atomic Transaction)

Энэхүү функц нь хүргэгч захиалгыг хүлээн авах үед өгөгдлийн бүрэн бүтэн байдлыг хангах зорилгоор Firestore Transaction ашигладаг.

```
export const acceptOrder = async (orderId: string, courierId:
    string) => {
  try {
    await runTransaction(db, async (transaction) => {
      // 1. Get order ref
      const orderRef = doc(db, "orders", orderId);
      const orderDoc = await transaction.get(orderRef);

      if (!orderDoc.exists()) {
        throw "Order does not exist!";
      }

      // 2. Get courier info for denormalization
      const courierRef = doc(db, "users", courierId);
      const courierDoc = await transaction.get(courierRef);

      if (!courierDoc.exists()) {
        throw "Courier profile not found!";
      }

      const courierData = courierDoc.data();

      // 3. Prepare denormalized data
      const courierInfo: CourierInfo = {
        name: courierData.name || courierData.displayName || "
    Courier",
        phone: courierData.phone || courierData.phoneNumber || "",
      };

      if (courierData.photoURL) {
        courierInfo.photoURL = courierData.photoURL;
      }

      // 4. Update order atomically
      transaction.update(orderRef, {
        status: "accepted",
        courierId: courierId,
        acceptedAt: serverTimestamp(),
        updatedAt: serverTimestamp(),
        courierInfo: courierInfo
      });
    });

    console.log("Order accepted successfully via transaction");
  } catch (error) {
    console.error("Transaction failed: ", error);
    throw error;
  }
};
```

Listing 5: Accept Order Transaction

## A.2 Firestore Security Rules

Хэрэглэгчийн өгөгдлийг хамгаалах дүрмийн жишээ.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper functions
    function isAuthenticated() {
      return request.auth != null;
    }

    function isOwner(userId) {
      return request.auth.uid == userId;
    }

    // Users collection
    match /users/{userId} {
      allow read: if isAuthenticated();
      allow create: if isAuthenticated() && isOwner(userId);
      allow update: if isAuthenticated() && isOwner(userId);
    }

    // Orders collection
    match /orders/{orderId} {
      allow read: if isAuthenticated();
      allow create: if isAuthenticated();
      allow update: if isAuthenticated();
    }
  }
}
```

Listing 6: Firestore Security Rules

## A.3 Firebase тохиргоо (firebase.ts)

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";
import { getStorage } from "firebase/storage";

const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.
    EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};

const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const db = getFirestore(app);
export const storage = getStorage(app);
```

```
20
21 export default app;
```

Listing 7: Firebase Config